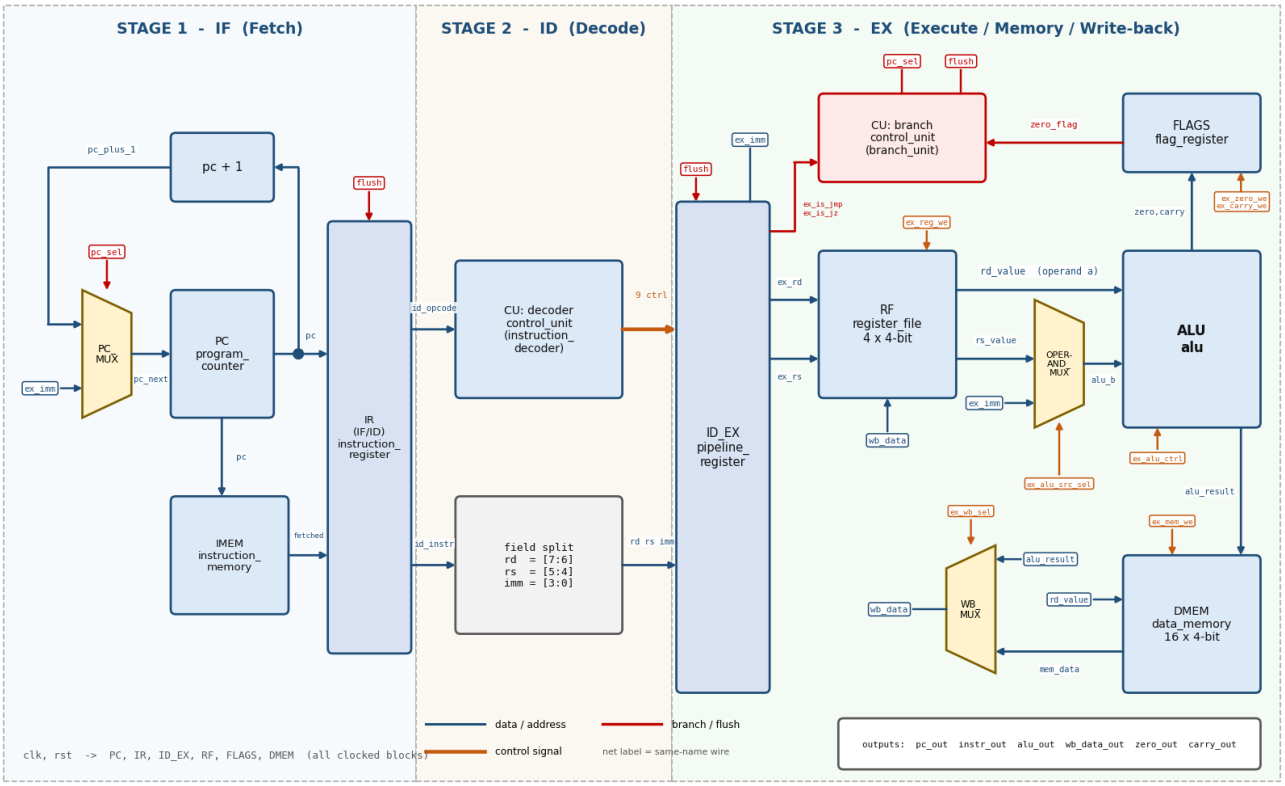
**PROJECT 02**

# DESIGN AND VERIFICATION OF A 4-BIT 3-STAGE PIPELINED CPU IN VERILOG




**Student:** Truong Dong Hung
**Roll No:** SE201121

**Ho Chi Minh City, October 2026**

# Abstract

This report presents a **4-bit CPU with a three-stage pipeline** (Fetch, Decode, Execute) written in Verilog HDL and simulated in Xilinx Vivado. The processor has a 12-bit instruction word, twelve instructions (ADD, SUB, AND, OR, XOR, NOT, LDI, MOV, LOAD, STORE, JMP, JZ), four 4-bit registers, Zero and Carry flags, and separate 16-word instruction and data memories. One instruction completes in every clock cycle; a taken jump is resolved in the Execute stage and costs two cycles because the two younger instructions are flushed. The design consists of eleven Verilog files and is verified by self-checking testbenches that print the student name and roll number, run an examples part and a checking part, and end with a PASS line (88 checks in total: 11 ALU, 11 register file, 22 control unit and 44 CPU). The report gives the instruction set, the block and top-level diagrams, the main truth tables, the Vivado flow with screenshots F1 to F6, and the simulation waveforms.

# Contents

Abstract ..... 2
1. Introduction ..... 4
1.1. Objectives and Scope ..... 4
2. System Design ..... 5
2.1. Instruction Set ..... 5
2.2. Block Diagram and Pipeline ..... 5
2.3. Top-Level Diagram ..... 7
2.4. Modules and Control Signals ..... 8
2.5 File Hierarchy ..... 8
3. Truth Tables from Simulation ..... 10
4. Vivado Flow ..... 11
4.2. Utilization ..... 15
5. Design, Truth Tables, Waveforms and Source Code ..... 16
5.1. mux2 - 2-Input, 4-Bit Multiplexer ..... 17
5.2. program_counter - Program Counter ..... 19
5.3. instruction_memory - Program ROM ..... 21
5.4. instruction_register - IF/ID Pipeline Register ..... 23
5.5. control_unit - Decoder and Branch Unit ..... 26
5.6. pipeline_register - ID/EX Pipeline Register ..... 32
5.7. register_file - Register File ..... 35
5.8. alu - Arithmetic Logic Unit ..... 38
5.9. flag_register - Zero and Carry Flags ..... 42
5.10. data_memory - Data RAM ..... 44
5.11. cpu_top - Top-Level Module ..... 46
6. Testbench and Simulation Results ..... 52
6.1. Testbench Structure ..... 52
6.2. Waveforms of the CPU Testbench (F2) ..... 52
6. Conclusion ..... 54

# 1. Introduction

A CPU fetches an instruction, decodes it and executes it. In a **pipelined** CPU the three steps overlap, so that while one instruction is executed the next one is decoded and the third one is fetched. This project designs such a processor in Verilog, shows how each module works, and verifies it with testbenches and Vivado simulation.

## 1.1. Objectives and Scope

- Define a small instruction set (12 instructions, 12-bit word) and build a 3-stage pipelined CPU in modular Verilog.
- Handle pipeline control: flush after a taken jump, flags that behave correctly, and no stall for dependent instructions.
- Verify the design with self-checking testbenches and show the results with Vivado screenshots (F1 to F6).

| Item | Value |
|---|---|
| **Language / tool** | Verilog HDL; Xilinx Vivado (XSim simulator); target part xc7k70tfbv676-1 |
| **Data / instruction width** | 4-bit data; 12-bit instruction: opcode [11:8], Rd [7:6], Rs [5:4], immediate [3:0] |
| **Registers / flags** | R0 to R3 (4 bits each); Zero and Carry |
| **Memories** | Instruction ROM 16 x 12 bits; data RAM 16 x 4 bits (Harvard) |
| **Pipeline** | IF, ID, EX; jump decided in EX; 2-cycle penalty; no data hazards |
| **Verification** | Self-checking testbenches, 88 checks, name and roll number printed, PASS line at the end |

*Table 1. Project summary*

# 2. System Design

## 2.1. Instruction Set

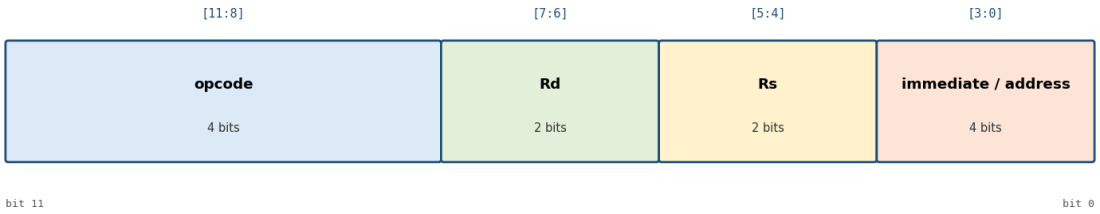



*Figure 1: Instruction format (12 bits).*

| Opcode | Instruction | Operation | Flags set |
|---|---|---|---|
| 0000 | ADD Rd,Rs | Rd ← Rd + Rs | Z, C |
| 0001 | SUB Rd,Rs | Rd ← Rd - Rs (C = 1 when no borrow) | Z, C |
| 0010 | AND Rd,Rs | Rd ← Rd AND Rs | Z |
| 0011 | OR Rd,Rs | Rd ← Rd OR Rs | Z |
| 0100 | XOR Rd,Rs | Rd ← Rd XOR Rs | Z |
| 0101 | NOT Rd | Rd ← NOT Rd | Z |
| 0110 | LDI Rd,imm | Rd ← imm | - |
| 0111 | MOV Rd,Rs | Rd ← Rs | - |
| 1000 | LOAD Rd,addr | Rd ← DMEM[addr] | - |
| 1001 | STORE Rd,addr | DMEM[addr] ← Rd | - |
| 1010 | JMP addr | PC ← addr | - |
| 1011 | JZ addr | if Z = 1 then PC ← addr | - |
| 1100 - 1111 | reserved | NOP | - |

*Table 2. Instruction set*

## 2.2. Block Diagram and Pipeline

Figure 2 shows the functional blocks by stage. The **program counter** addresses the instruction memory; the **IF/ID register** stores the instruction; the **control unit** decodes it; the **ID/EX register** passes the control bits to the Execute stage, where the **register file**, **ALU**, **flag register** and **data memory** do the work and the **branch unit** decides jumps. A jump sends `pc_sel` and `flush` (red) back to the PC and the pipeline registers.

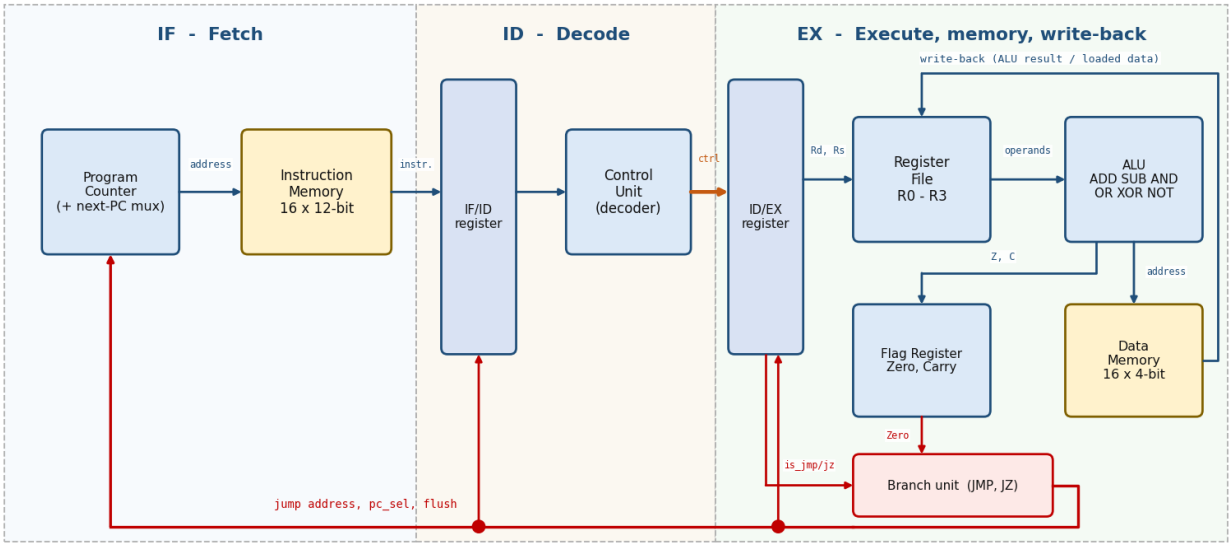



*Figure 2: Block diagram of the 4-bit pipelined CPU.*

| Hazard | Cause | Solution |
|---|---|---|
| Structural | Fetch and LOAD/STORE need memory in the same cycle | Separate instruction and data memories (Harvard) |
| Data | An instruction needs the result of the previous one | Registers are read and written in the same stage (EX); the next instruction reads after the write edge, so no stall and no forwarding |
| Control | A jump is known in EX, when 2 younger instructions are already in the pipeline | Branch unit raises `flush`: IF/ID and ID/EX are loaded with bubbles and the PC loads the jump address (2-cycle penalty) |

*Table 3. Hazards and how the design handles them*

## 2.3. Top-Level Diagram

Figure 4 is the top-level diagram of `cpu_top`. Boxes are the instances of the Verilog source and signal names are those of `cpu_top.v`; wires that travel far (`pc_sel`, `flush`, `ex_imm`, `wb_data`) are drawn with matching net labels.

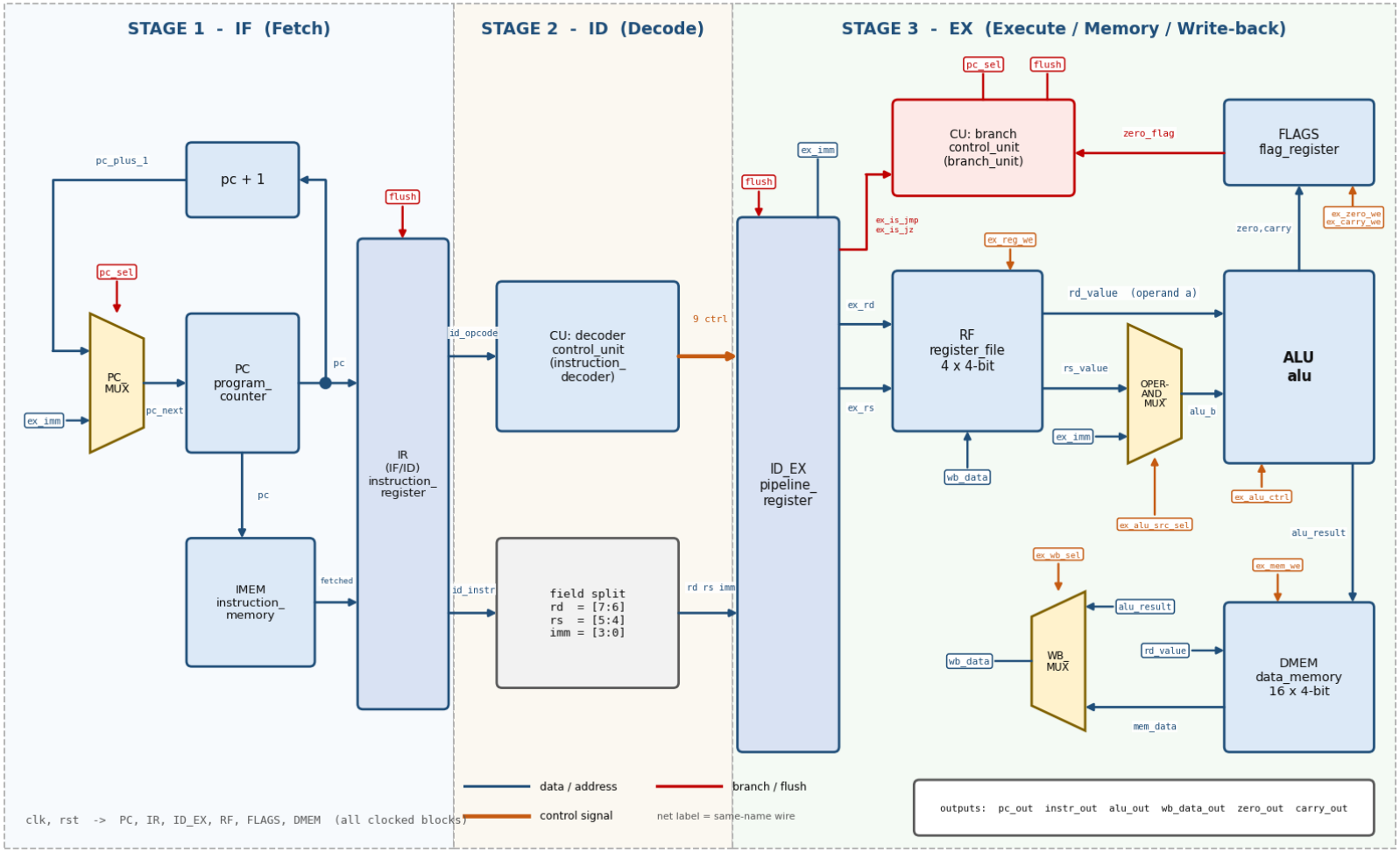



*Figure 4: Top-level diagram of cpu_top.*

## 2.4. Modules and Control Signals

| Instance | Module (file) | Function |
|---|---|---|
| PC | program_counter | holds the address of the instruction being fetched (0 to 15) |
| PC_MUX | mux2 (mux.v) | chooses the next address: PC + 1 or the jump address |
| IMEM | instruction_memory | ROM 16 x 12 bits with the program, read from a .mem file |
| IR | instruction_register | IF/ID pipeline register; loads a bubble on reset or flush |
| CU | control_unit | decoder (ID) turns the opcode into control bits; branch unit (EX) decides jumps |
| ID_EX | pipeline_register | ID/EX pipeline register; carries control bits and instruction fields to EX |
| RF | register_file | R0 to R3, two read ports, one clocked write port |
| OPERAND_MUX | mux2 | ALU input B: register Rs or immediate |
| ALU | alu | ADD, SUB, AND, OR, XOR, NOT, PASS with carry and zero outputs |
| FLAGS | flag_register | stores Zero and Carry with separate enables |
| DMEM | data_memory | RAM 16 x 4 bits for LOAD and STORE |
| WB_MUX | mux2 | write-back value: ALU result or memory data |

*Table 4. The 12 instances of cpu_top and their function*

| Signal | Made in | Used in | Meaning |
|---|---|---|---|
| alu_ctrl (3 bits) | ID | EX | ALU operation (0 ADD, 1 SUB, 2 AND, 3 OR, 4 XOR, 5 NOT, 6 PASS) |
| alu_src_sel | ID | EX | Operand MUX: 0 = Rs, 1 = immediate |
| wb_sel | ID | EX | Write-back MUX: 0 = ALU result, 1 = memory data |
| reg_we | ID | EX | Write the destination register Rd |
| mem_we | ID | EX | Write the data memory |
| zero_we, carry_we | ID | EX | Update the Zero / Carry flag |
| is_jmp, is_jz | ID | EX | Tell the branch unit what kind of jump this is |
| pc_sel | EX (branch unit) | IF | PC MUX: 0 = PC + 1, 1 = jump address; pc_sel = is_jmp OR (is_jz AND Zero) |
| flush | EX (branch unit) | IF/ID, ID/EX | Empty both pipeline registers |
| pc_en, ir_load | fixed at 1 | IF | Fetch a new instruction every clock |

*Table 5. Control signals: where they are made, used, and what they mean*

## 2.5 File Hierarchy

The design is split into eleven source files. Figure 6 shows the instantiation tree and Table 9 lists every instance with its module and file. `cpu_top.v` instantiates ten modules (the multiplexer module three times) and contains only wiring and one adder (`pc + 1`). The two composite modules `alu` and `control_unit` contain their own sub-modules inside the same file.

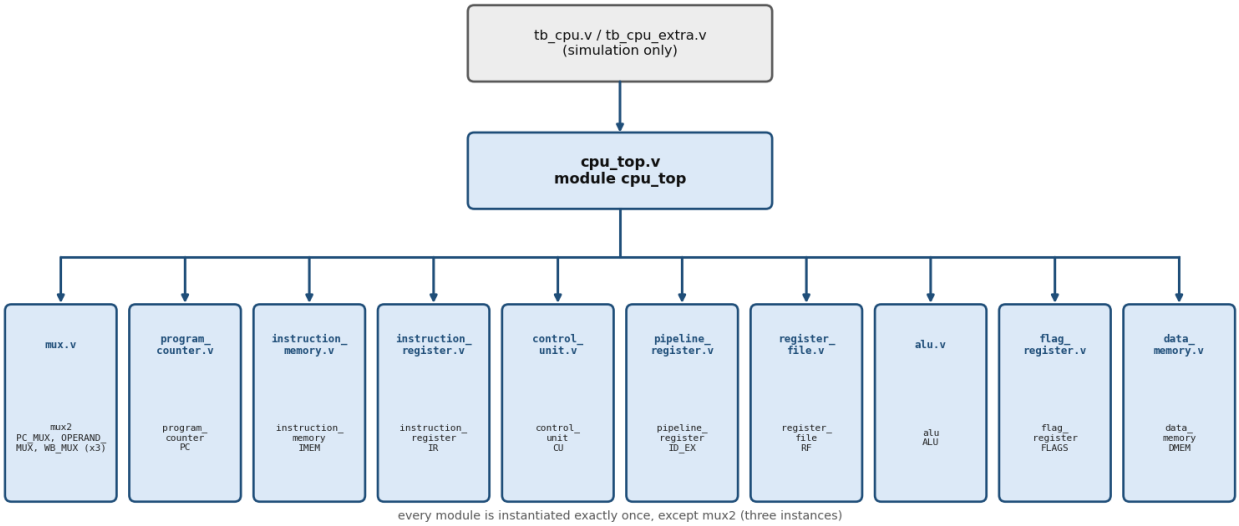



*Figure 5: File hierarchy of the 4-bit CPU*

# 3. Truth Tables from Simulation

The tables below were obtained by simulating the modules and show their exact behaviour.

| opcode | Op | alu_ctrl | src | wb | reg_we | mem_we | zero_we | carry_we | jmp | jz |
|---|---|---|---|---|---|---|---|---|---|---|
| 0000 | ADD | 0 | 0 | 0 | 1 | 0 | 1 | 1 | 0 | 0 |
| 0001 | SUB | 1 | 0 | 0 | 1 | 0 | 1 | 1 | 0 | 0 |
| 0010 | AND | 2 | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 0 |
| 0011 | OR | 3 | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 0 |
| 0100 | XOR | 4 | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 0 |
| 0101 | NOT | 5 | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 0 |
| 0110 | LDI | 6 | 1 | 0 | 1 | 0 | 0 | 0 | 0 | 0 |
| 0111 | MOV | 6 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 |
| 1000 | LOAD | 6 | 1 | 1 | 1 | 0 | 0 | 0 | 0 | 0 |
| 1001 | STORE | 6 | 1 | 0 | 0 | 1 | 0 | 0 | 0 | 0 |
| 1010 | JMP | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 |
| 1011 | JZ | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 |
| 1100 | NOP | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 1101 | NOP | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 1110 | NOP | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 1111 | NOP | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

*Table 6. Instruction decoder: control bits for all 16 opcodes*

| alu_ctrl | Operation | a | b | result | carry | zero |
|---|---|---|---|---|---|---|
| 0 | ADD | 0101 | 0011 | 1000 | 0 | 0 |
| 0 | ADD | 1111 | 0001 | 0000 | 1 | 1 |
| 0 | ADD | 1000 | 1000 | 0000 | 1 | 1 |
| 1 | SUB | 0101 | 0011 | 0010 | 1 | 0 |
| 1 | SUB | 0011 | 0101 | 1110 | 0 | 0 |
| 1 | SUB | 0111 | 0111 | 0000 | 1 | 1 |
| 1 | SUB | 0000 | 0001 | 1111 | 0 | 0 |
| 2 | AND | 1010 | 0101 | 0000 | 0 | 1 |
| 3 | OR | 1100 | 1010 | 1110 | 0 | 0 |
| 4 | XOR | 1100 | 1010 | 0110 | 0 | 0 |
| 5 | NOT | 0101 | 1111 | 1010 | 0 | 0 |
| 5 | NOT | 1111 | 0000 | 0000 | 0 | 1 |
| 6 | PASS | 1111 | 1001 | 1001 | 0 | 0 |
| 7 | reserved | 1010 | 0101 | 0000 | 0 | 1 |

*Table 7. ALU: results and flags (a, b in binary)*

| ex_is_jmp | ex_is_jz | zero_flag | pc_sel = flush | Meaning |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | no jump |
| 0 | 0 | 1 | 0 | no jump |
| 0 | 1 | 0 | 0 | JZ not taken |
| 0 | 1 | 1 | 1 | JZ taken |
| 1 | 0 | 0 | 1 | JMP: always taken |
| 1 | 0 | 1 | 1 | JMP: always taken |

*Table 8. Branch unit: when a jump is taken*

# 4. Vivado Flow

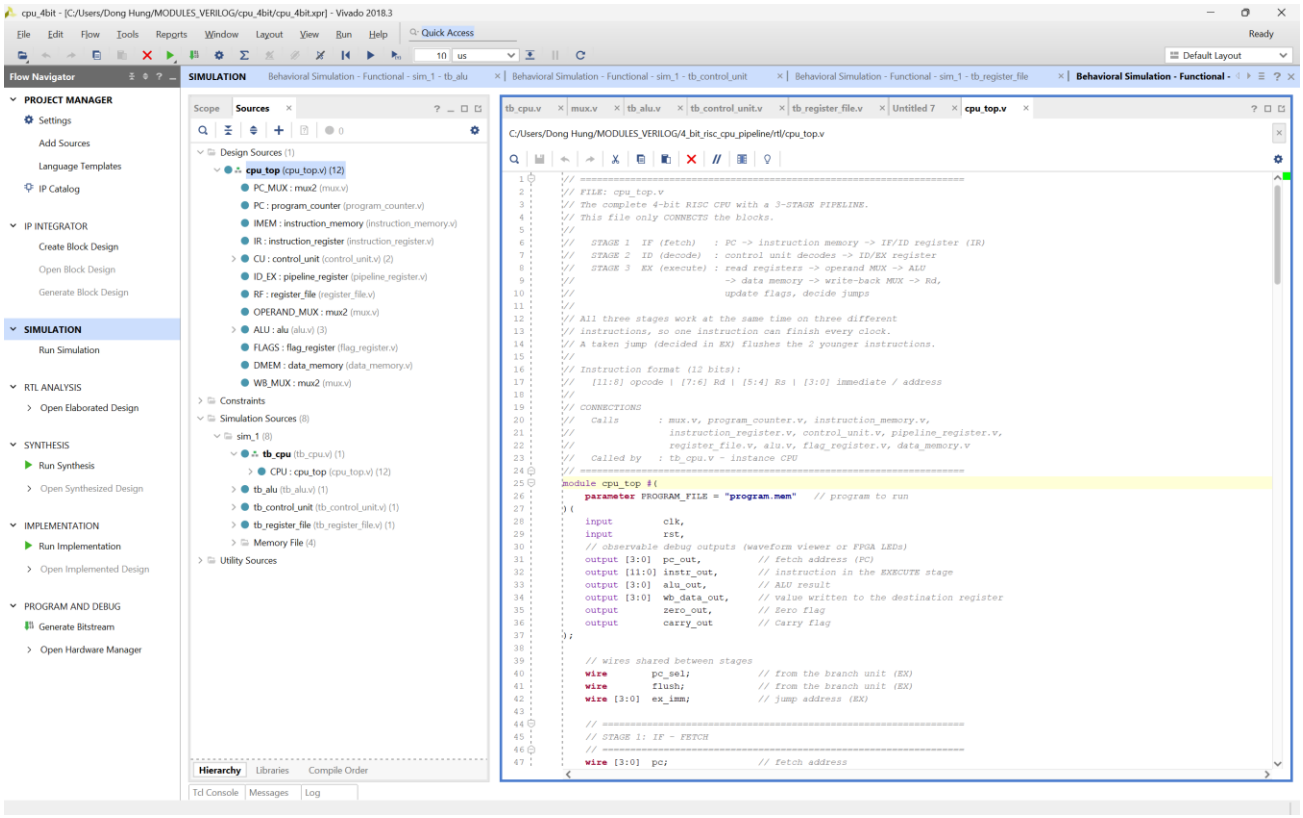


*Figure F1: Sources window.*

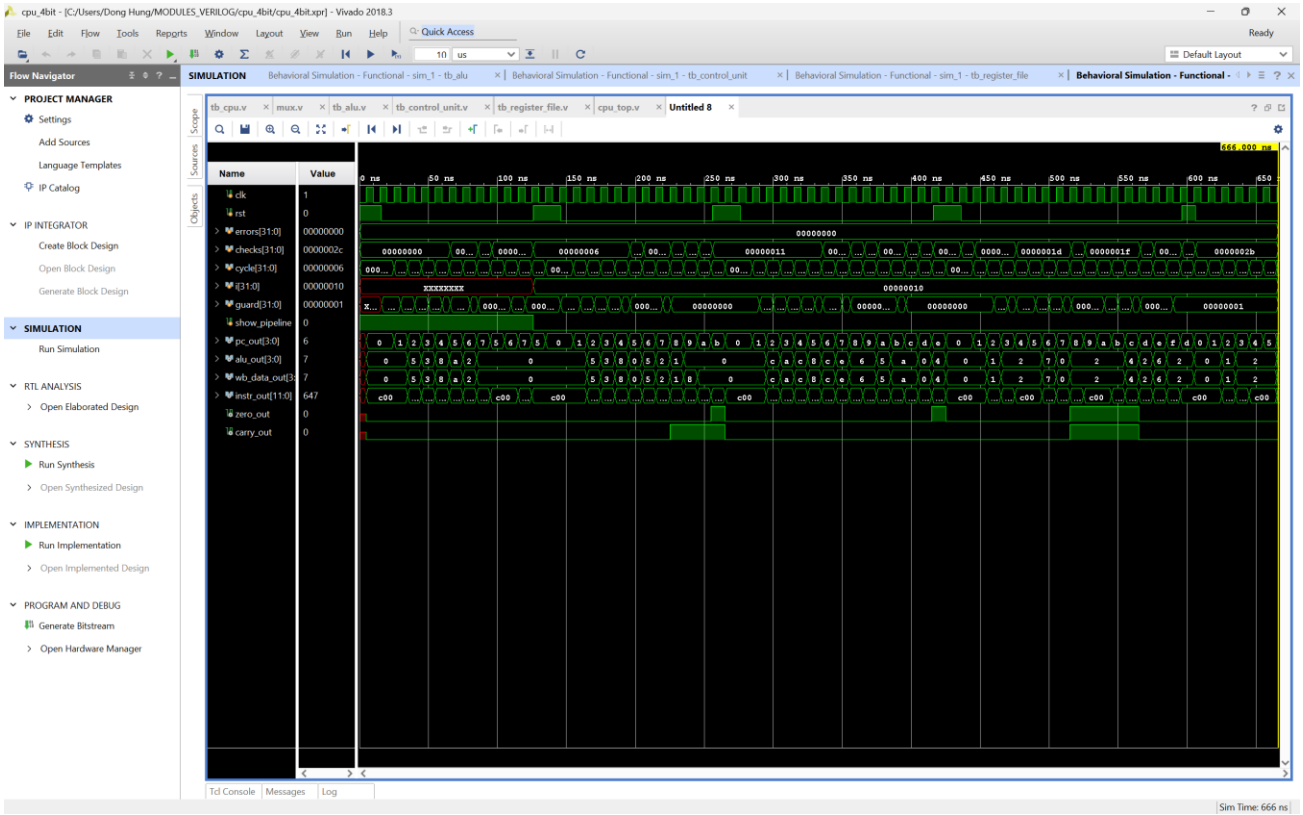


*Figure F2: Behavioural simulation waveform.*

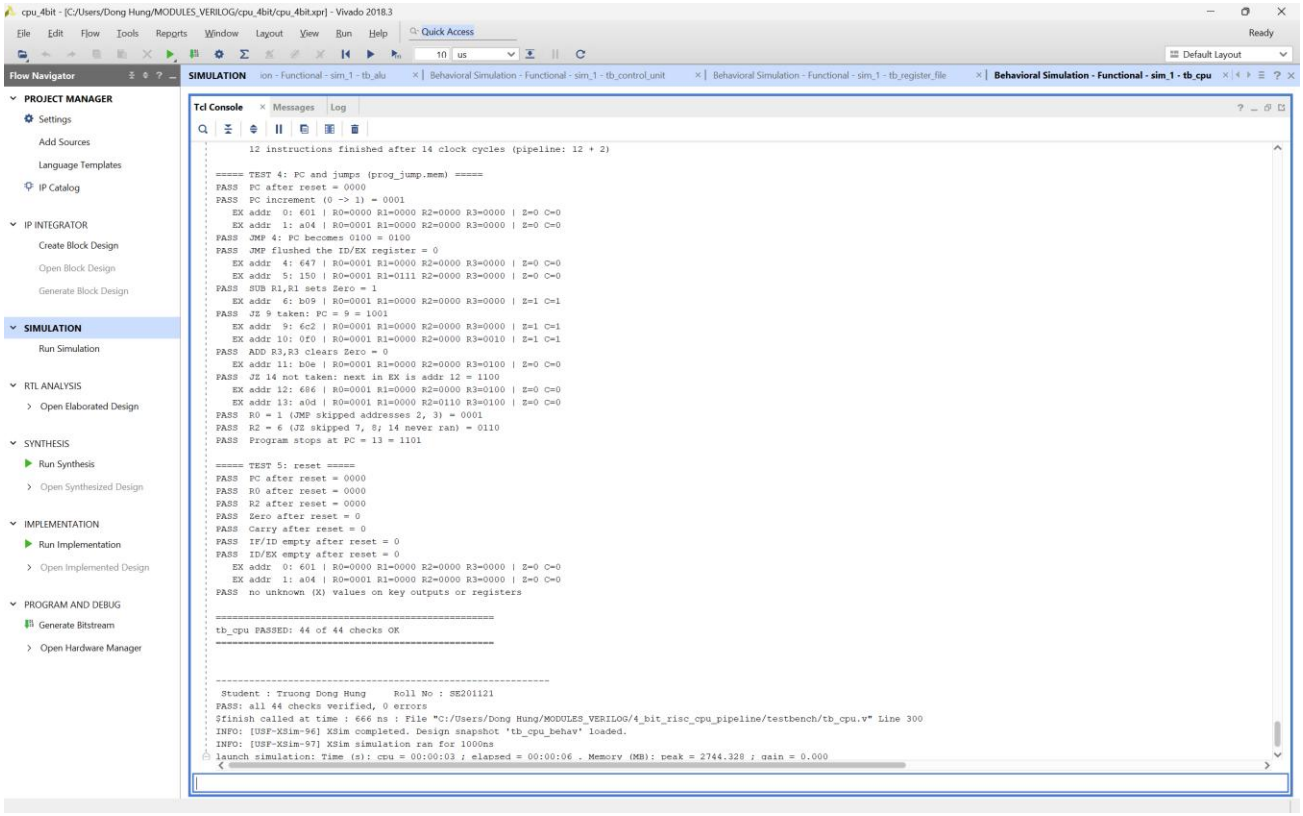



*Figure F3: Tcl Console.*

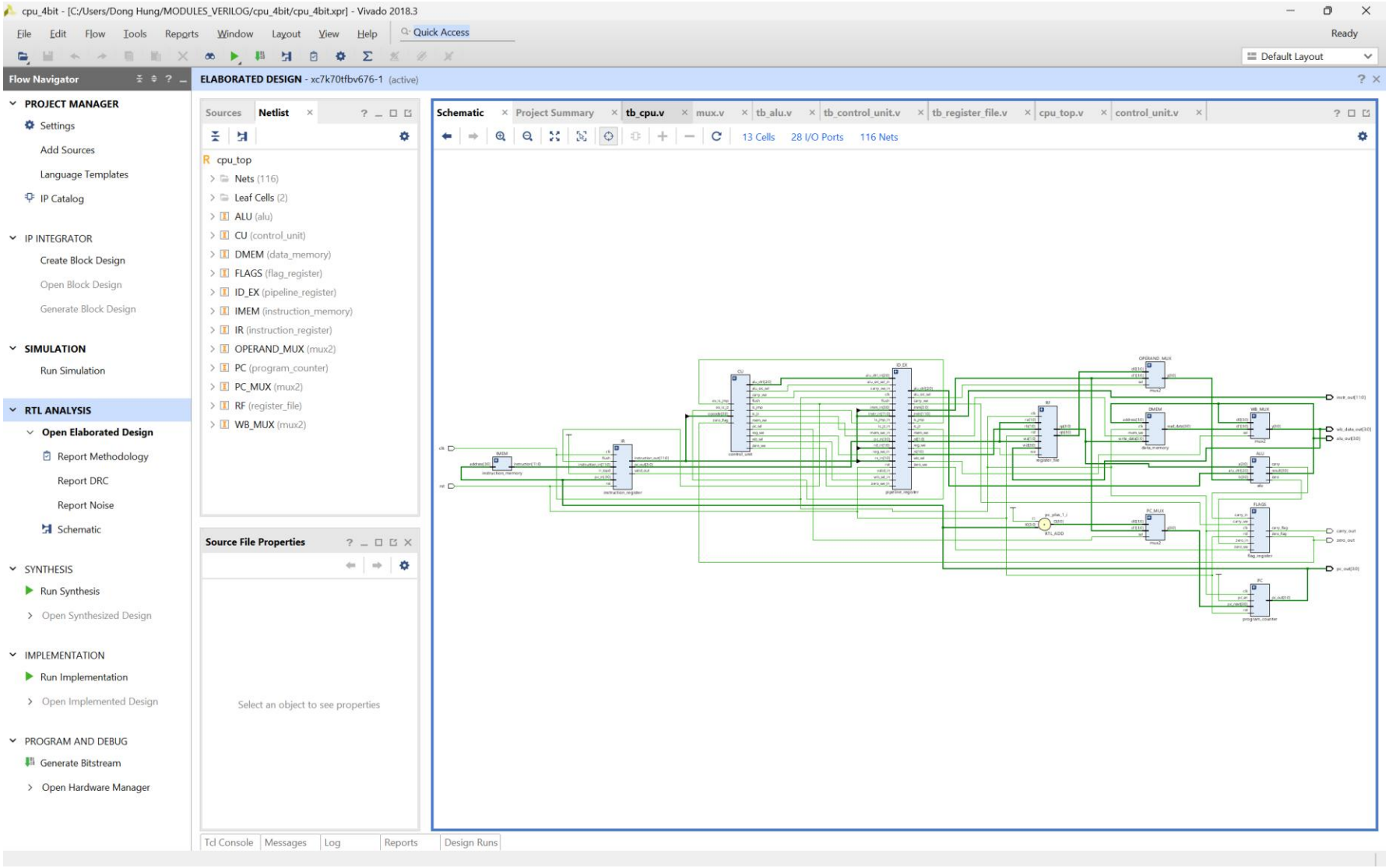



*Figure F4: Elaborated schematic of cpu_top (RTL Analysis).*

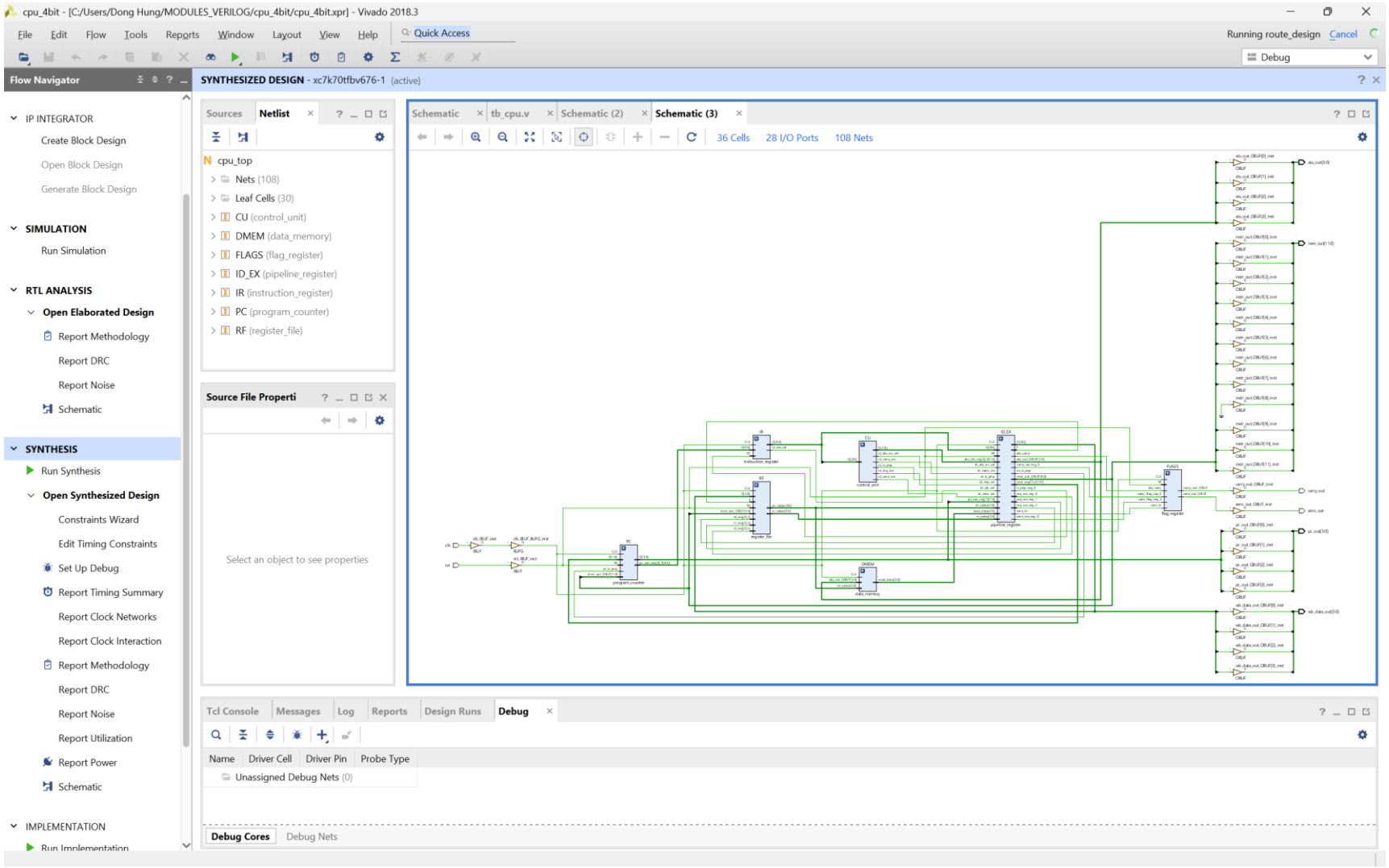



*Figure F5: Synthesized schematic of cpu_top (after Run Synthesis)*

## 4.2. Utilization

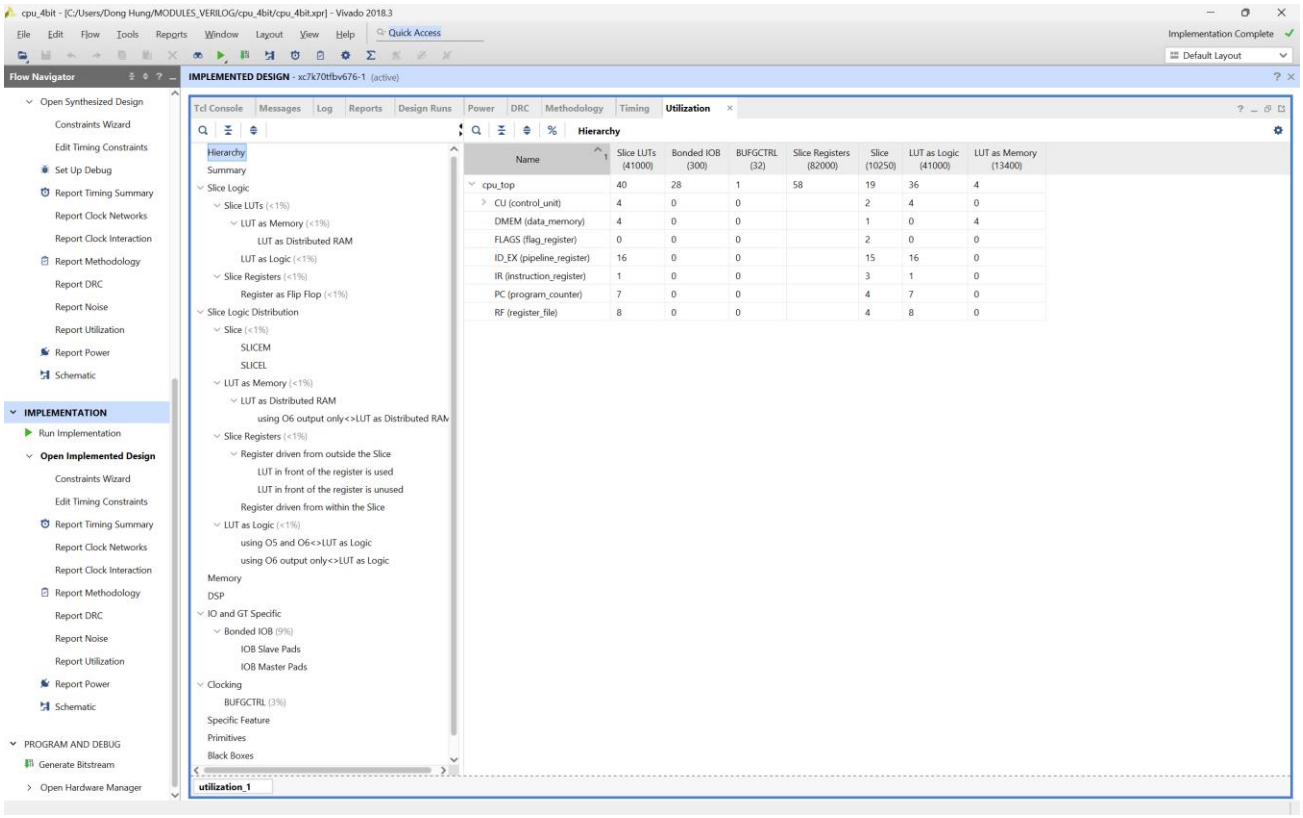



*Figure F6: Utilization report of the synthesized design*

# 5. Design, Truth Tables, Waveforms and Source Code

Each module below follows the same five-part structure: **(1) function**, what the module does and why it exists, with its port list; **(2) block diagram**, the internal structure; **(3) truth table**, the values observed when the module is simulated; **(4) waveform**, a simulation result that shows the same behaviour on real signals; and **(5) source code**, the complete Verilog of the file. All truth tables and waveforms in this chapter were produced by simulating exactly the Verilog source that is printed next to them.

## 5.1. mux2 - 2-Input, 4-Bit Multiplexer

**Function:** `mux2` selects one of two 4-bit inputs: y = d1 when sel = 1, otherwise y = d0. The CPU needs this selection three times, so it is written once and instantiated as **PC_MUX** (pc + 1 or the jump address), **OPERAND_MUX** (register Rs or the immediate) and **WB_MUX** (ALU result or memory data).

| Port | Direction | Description |
|---|---|---|
| d0 [3:0] | input | selected when sel = 0 |
| d1 [3:0] | input | selected when sel = 1 |
| sel | input | select line |
| y [3:0] | output | selected value (combinational) |

*Table 11. mux2 - port list*

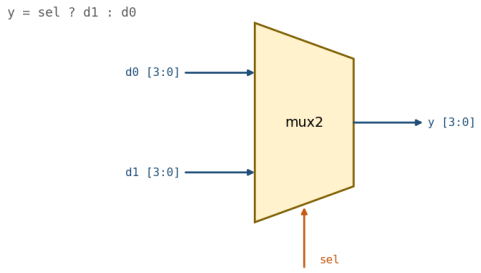



*Figure 7: Block diagram of mux2.*

| sel | d0 | d1 | y |
|---|---|---|---|
| 0 | 0101 | 1010 | 0101 |
| 0 | 0011 | 1100 | 0011 |
| 1 | 0101 | 1010 | 1010 |
| 1 | 0011 | 1100 | 1100 |

*Table 12. mux2 - truth table from simulation*

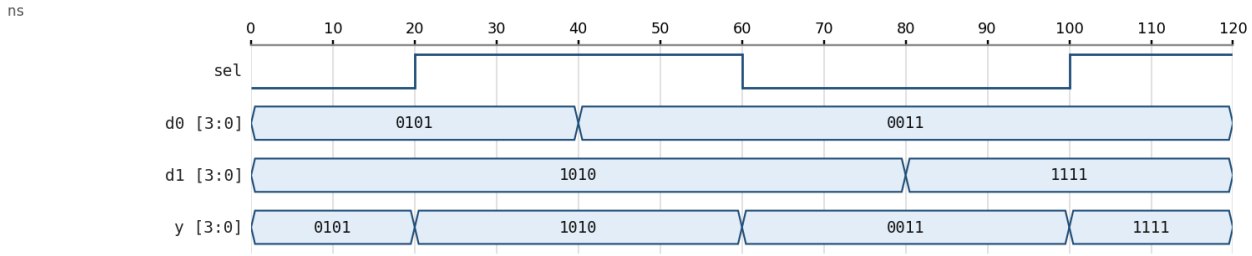



*Figure 8: Waveform of mux2.*

Waveform: with sel = 0 the output follows d0 (0101). At 20 ns sel becomes 1 and y switches to d1 (1010). Changing d0 at 40 ns has no effect while sel = 1; at 60 ns sel returns to 0 and y shows the new d0 (0011). Changing d1 at 80 ns is ignored until sel = 1 again at 100 ns, when y becomes 1111. The output reacts immediately because the module is purely combinational.

```
// ======================================================================
// FILE: mux.v
// 2-input, 4-bit multiplexer. A mux picks ONE of its inputs.
//   sel = 0 -> y = d0
//   sel = 1 -> y = d1
//
// The CPU uses this module three times:
//   1. Operand MUX    : ALU input B = register Rs  or immediate
```

```
//   2. Write-back MUX : register data = ALU result or memory data
//   3. PC MUX         : next PC = PC + 1  or jump address
//
// CONNECTIONS
//   Calls       : nothing (no other modules)
//   Called by   : cpu_top.v - 3 times: instances PC_MUX, OPERAND_MUX, WB_MUX
// ================================================================
module mux2 (
    input  [3:0] d0,
    input  [3:0] d1,
    input        sel,
    output [3:0] y
);
    assign y = (sel == 1'b1) ? d1 : d0;
endmodule
```

***Source code: mux.v***

## 5.2. program_counter - Program Counter

**Function:** the program counter holds the address (0 to 15) of the instruction that is being fetched. On each rising clock edge it behaves as follows: if `rst` = 1 it is cleared to 0; otherwise, if `pc_en` = 1 it loads `pc_next`; otherwise it keeps its value. In this pipeline a new instruction is fetched every clock cycle, so `cpu_top` ties `pc_en` to 1. `pc_next` comes from PC_MUX and is either PC + 1 or the jump address from the EX stage. Since the counter is 4 bits wide, PC + 1 wraps from 15 to 0.

| Port | Direction | Description |
|---|---|---|
| clk | input | clock, rising edge |
| rst | input | synchronous reset, PC <- 0 |
| pc_en | input | 1 = load pc_next, 0 = hold |
| pc_next [3:0] | input | address to load |
| pc_out [3:0] | output | current fetch address (register output) |

*Table 13. program_counter - port list*

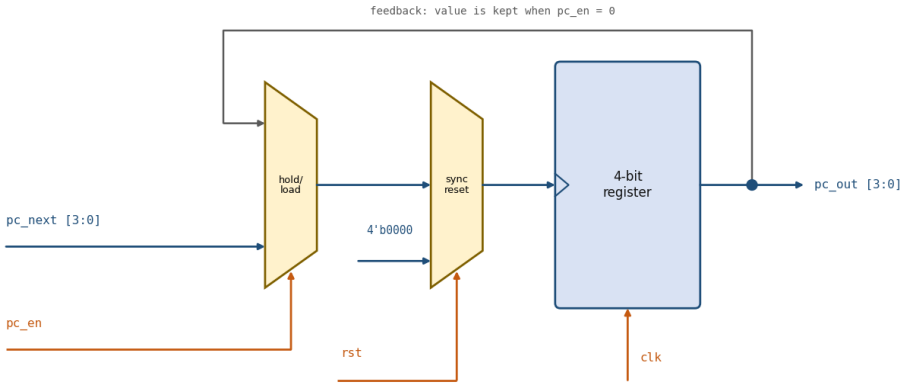



*Figure 9: Block diagram of program_counter.*

| rst | pc_en | pc_next | pc_out before | pc_out after | Action |
|---|---|---|---|---|---|
| 1 | 1 | 1010 | 0011 | 0000 | reset to 0 |
| 1 | 0 | 1010 | 0011 | 0000 | reset to 0 |
| 0 | 1 | 1010 | 0011 | 1010 | load pc_next |
| 0 | 0 | 1010 | 0011 | 0011 | hold |

*Table 14. program_counter - behaviour at one clock edge (PC = 3 before, pc_next = 1010)*

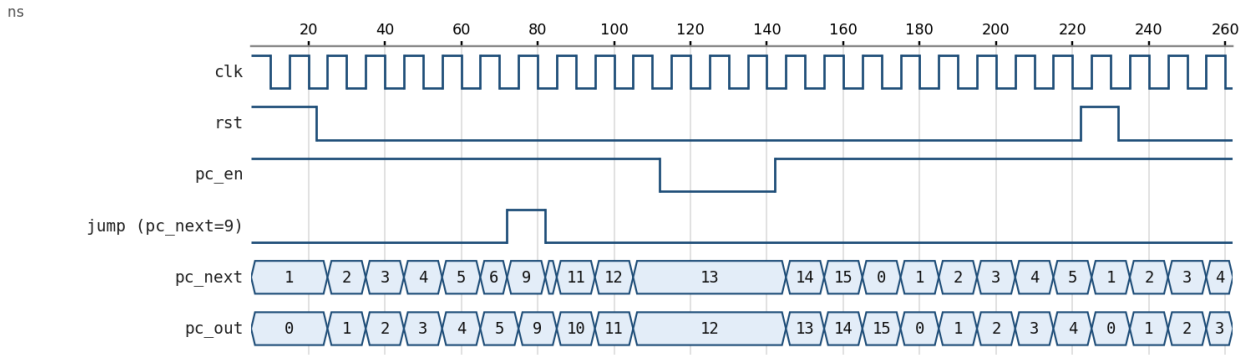



*Figure 10: Waveform of program_counter (pc_next = jump ? 9 : pc_out + 1).*

Waveform: `pc_out` stays 0 while `rst` is 1 and starts counting at the first edge after the reset is released (25 ns). At 75 ns the `jump` input is 1, so the counter loads 9 instead of 6 and continues with 10, 11, 12. From 112 ns to

142 ns pc_en is 0 and the counter holds 12 for three clock edges. It then counts 13, 14, 15 and wraps to 0 at 175 ns. A second reset at 222 ns forces the counter back to 0 at the next edge.

```
// ======================================================================
// FILE: program_counter.v
// The PC holds the address of the instruction being FETCHED (0 to 15).
//
// Interface (as in the project plan):
//   clk, rst, pc_en, pc_next, pc_out
//
// In the 3-stage pipeline a new instruction is fetched every clock,
// so pc_en is kept at 1 by cpu_top. pc_next is PC + 1, or the jump
// address when a jump is taken in the EX stage.
//
// CONNECTIONS
//   Calls       : nothing (no other modules)
//   Called by   : cpu_top.v - instance PC
// ======================================================================
module program_counter (
    input            clk,
    input            rst,
    input            pc_en,       // 1 = load pc_next
    input      [3:0] pc_next,     // the next address
    output reg [3:0] pc_out       // current fetch address
);
    always @(posedge clk) begin
        if (rst)
            pc_out <= 4'd0;       // after reset, start at address 0
        else if (pc_en)
            pc_out <= pc_next;    // go to the next address
        // if pc_en = 0, pc_out keeps its old value
    end
endmodule
```

***Source code: program_counter.v***

## 5.3. instruction_memory - Program ROM

**Function:** the instruction memory stores the program as 16 words of 12 bits. At time zero every word is filled with `C00` (reserved opcode, i.e. NOP), and then the program file named by the parameter `PROGRAM_FILE` is read with `$readmemh`, one hexadecimal instruction per line. The read is combinational: the instruction at `address` appears at the output without a clock. Because the memory is separate from the data memory, an instruction fetch never conflicts with a LOAD or STORE.

| Port | Direction | Description |
|---|---|---|
| address [3:0] | input | word to read (the PC) |
| instruction [11:0] | output | instruction at that address |
| PROGRAM_FILE | parameter | name of the hexadecimal program file |

*Table 15. instruction_memory - port list*

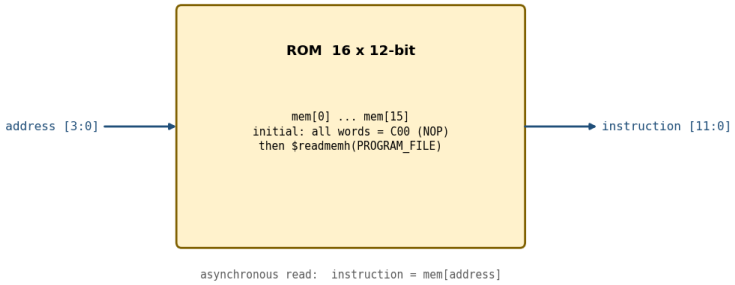



*Figure 11: Block diagram of instruction_memory.*

| Address | Instruction (hex) | opcode \| Rd \| Rs \| imm | Assembly |
|---|---|---|---|
| 0 | 645 | 0110 \| 01 \| 00 \| 0101 | LDI R1,5 |
| 1 | 683 | 0110 \| 10 \| 00 \| 0011 | LDI R2,3 |
| 2 | 060 | 0000 \| 01 \| 10 \| 0000 | ADD R1,R2 |
| 3 | 6CA | 0110 \| 11 \| 00 \| 1010 | LDI R3,10 |
| 4 | 470 | 0100 \| 01 \| 11 \| 0000 | XOR R1,R3 |
| 5 | A05 | 1010 \| 00 \| 00 \| 0101 | JMP 5 |
| 6 | C00 | 1100 \| 00 \| 00 \| 0000 | NOP |
| 7 | C00 | 1100 \| 00 \| 00 \| 0000 | NOP |

*Table 16. instruction_memory - contents of program.mem read from simulation*

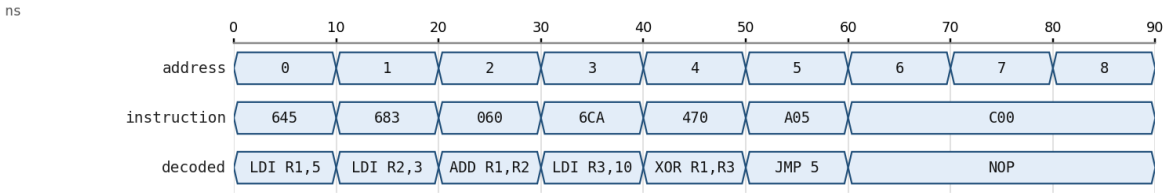



*Figure 12: Waveform of instruction_memory with program.mem.*

Waveform: the address is stepped from 0 to 8 every 10 ns. The instruction output follows the address after zero delay and shows the six programmed words (645, 683, 060, 6CA, 470, A05) followed by the default NOP word C00 for the unused addresses 6, 7 and 8.

```
// ======================================================================
// FILE: instruction_memory.v
// Stores the program: 16 words x 12 bits (an initialised ROM).
```

```
//
// Interface (as in the project plan):
//   address[3:0], instruction[11:0]
//
// The program is read from a text file (default: program.mem).
// Each line of the file is one instruction in hex, e.g. 645
//
// CONNECTIONS
//   Calls       : nothing (built-in $readmemh only)
//   Called by   : cpu_top.v - instance IMEM
//                 tb_cpu.v - task load_program writes CPU.IMEM.mem
// ============================================================================
module instruction_memory #(
    parameter PROGRAM_FILE = "program.mem"   // file with the program
)(
    input  [3:0]  address,        // from the PC
    output [11:0] instruction     // instruction at that address
);
    reg [11:0] mem [0:15];        // 16 words, 12 bits each
    integer i;

    initial begin
        // fill every word with C00 first (C = reserved opcode = NOP),
        // so an empty location never contains unknown (X) values
        for (i = 0; i < 16; i = i + 1)
            mem[i] = 12'hC00;
        // then load the program from the file
        $readmemh(PROGRAM_FILE, mem);
    end

    // read: output the instruction at the given address
    assign instruction = mem[address];
endmodule
```

***Source code: instruction_memory.v***

## 5.4. instruction_register - IF/ID Pipeline Register

**Function:** this module is the pipeline register between the IF and ID stages. At every clock edge with `ir_load` = 1 it stores the fetched instruction together with its address and sets `valid_out` = 1. If `rst` or `flush` is 1 it stores a bubble instead (instruction C00, address 0, `valid_out` = 0); reset and flush have priority over the load. With `ir_load` = 0 the old content is kept. The `valid_out` bit distinguishes a bubble from a real NOP in the waveforms; both decode to the same harmless control bits.

| Port | Direction | Description |
|---|---|---|
| clk, rst | input | clock and synchronous reset |
| ir_load | input | 1 = capture the new instruction (tied to 1 in cpu_top) |
| flush | input | 1 = replace the content by a bubble (taken jump) |
| instruction_in [11:0] | input | word from the instruction memory |
| pc_in [3:0] | input | address of that instruction |
| instruction_out [11:0] | output | instruction in the ID stage |
| pc_out [3:0] | output | its address |
| valid_out | output | 1 = real instruction, 0 = bubble |

*Table 17. instruction_register - port list*

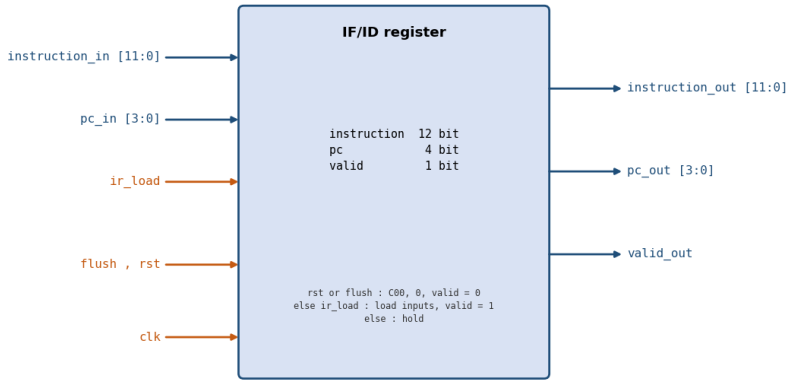



*Figure 13: Block diagram of instruction_register.*

| rst | flush | ir_load | instruction_out | pc_out | valid_out | Action |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 123 | 7 | 1 | hold |
| 0 | 0 | 1 | 645 | 5 | 1 | load |
| 0 | 1 | 0 | C00 | 0 | 0 | bubble |
| 0 | 1 | 1 | C00 | 0 | 0 | bubble |
| 1 | 0 | 0 | C00 | 0 | 0 | bubble |
| 1 | 0 | 1 | C00 | 0 | 0 | bubble |
| 1 | 1 | 0 | C00 | 0 | 0 | bubble |
| 1 | 1 | 1 | C00 | 0 | 0 | bubble |

*Table 18. instruction_register - result of one clock edge (old content 123 at address 7, new input 645 at address 5)*

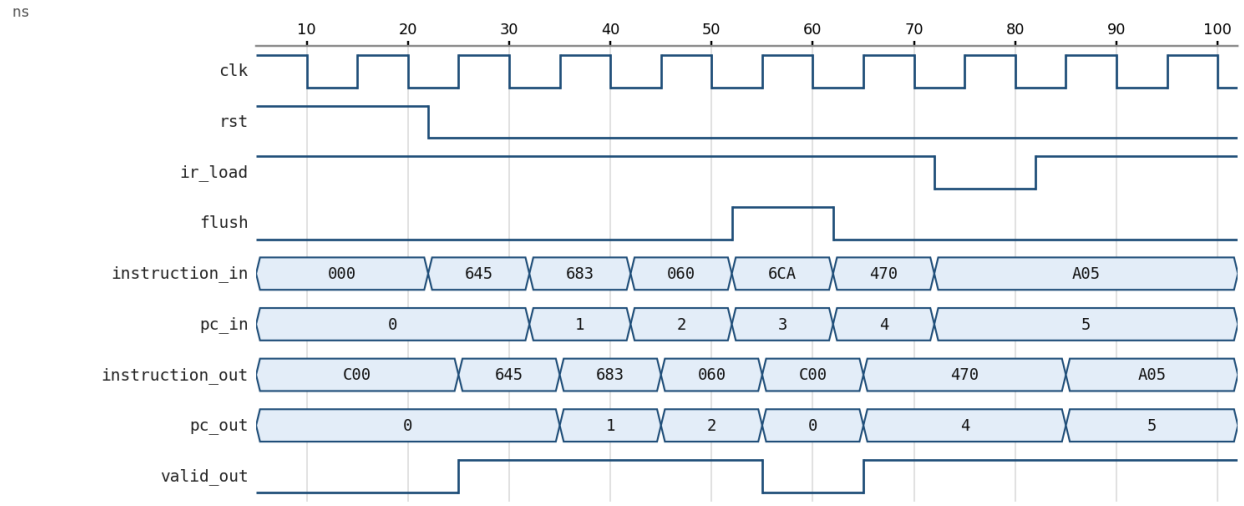



*Figure 14: Waveform of instruction_register.*

Waveform: while `rst` is 1 the output is the bubble C00 with `valid_out` = 0. At 25 ns, 35 ns and 45 ns the register loads 645, 683 and 060 together with their addresses 0, 1 and 2. At 55 ns `flush` is 1 while 6CA is offered at the input, so the output becomes C00 with `valid_out` = 0 and the offered instruction is lost. The next edge loads 470 normally. At 75 ns `ir_load` is 0 and the offered A05 is ignored (the output stays 470); when `ir_load` returns to 1, A05 is loaded at 85 ns.

```
// ========================================================================
// FILE: instruction_register.v
// The IR is the IF/ID PIPELINE REGISTER: it sits between the
// FETCH stage and the DECODE stage.
//
// Interface (as in the project plan):
//   clk, rst, ir_load, instruction_in, instruction_out
// Extra pipeline signals:
//   flush           : 1 = throw the instruction away (taken jump)
//   pc_in / pc_out  : address of the instruction (to follow it in the
//                     pipeline and in the waveform)
//   valid_out       : 1 = holds a real instruction, 0 = empty (bubble)
//
// Every clock (ir_load = 1) it saves the instruction just fetched.
//
// CONNECTIONS
//   Calls       : nothing (no other modules)
//   Called by   : cpu_top.v - instance IR (the IF/ID register)
// ========================================================================
module instruction_register (
    input             clk,
    input             rst,
    input             ir_load,          // 1 = save the new instruction
    input             flush,            // 1 = replace it with a NOP
    input      [11:0] instruction_in,   // from instruction memory
    input      [3:0]  pc_in,            // its address
    output reg [11:0] instruction_out,  // to the DECODE stage
    output reg [3:0]  pc_out,
    output reg        valid_out
);
    always @(posedge clk) begin
        if (rst || flush) begin
            instruction_out <= 12'hC00;     // C00 = reserved opcode = NOP
            pc_out          <= 4'd0;
            valid_out       <= 1'b0;        // empty slot (bubble)
        end
        else if (ir_load) begin
            instruction_out <= instruction_in;
```

```
            pc_out          <= pc_in;
            valid_out       <= 1'b1;
        end
    end
endmodule
```

***Source code: instruction_register.v***

## 5.5. control_unit - Decoder and Branch Unit

**Function:** `control_unit` is hardwired and contains two independent parts. The **instruction_decoder** works in the ID stage: it turns the 4-bit opcode into nine control bits, with a default of "do nothing" for every bit so that only the signals an instruction needs are switched on. The **branch_unit** works in the EX stage: it decides whether the jump in EX is taken, `pc_sel = ex_is_jmp OR (ex_is_jz AND zero_flag)`, and raises `flush` with the same value. The reserved opcodes 1100 to 1111 produce no active signal at all.

| Port | Direction | Description |
|---|---|---|
| opcode [3:0] | input | instruction bits [11:8] (ID stage) |
| alu_ctrl [2:0] | output | ALU operation code (0 ADD, 1 SUB, 2 AND, 3 OR, 4 XOR, 5 NOT, 6 PASS) |
| alu_src_sel | output | operand B: 0 = Rs, 1 = immediate |
| wb_sel | output | write-back: 0 = ALU result, 1 = memory data |
| reg_we, mem_we | output | write enable of the register file / data memory |
| zero_we, carry_we | output | update enables of the Zero / Carry flag |
| is_jmp, is_jz | output | instruction is JMP / JZ |
| ex_is_jmp, ex_is_jz | input | jump type of the instruction in EX (from ID_EX) |
| zero_flag | input | saved Zero flag |
| pc_sel | output | 1 = load the jump address into the PC |
| flush | output | 1 = empty IF/ID and ID/EX (equal to pc_sel) |

*Table 19. control_unit - port list*

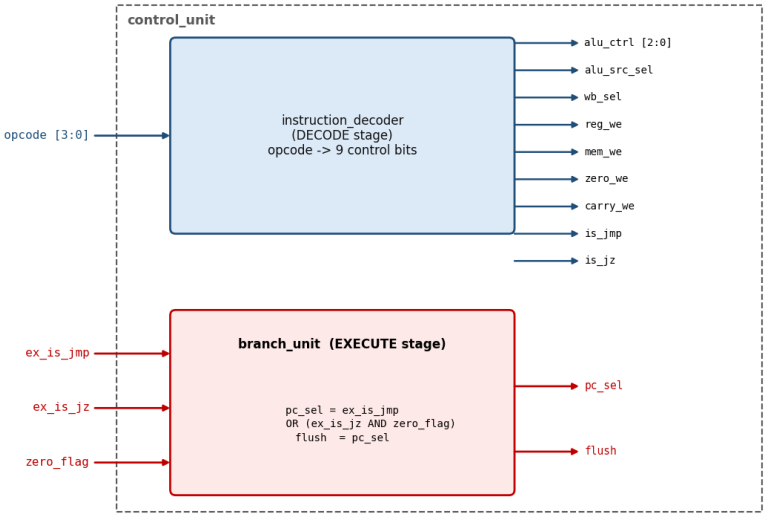



*Figure 15: Block diagram of control_unit.*

| opcode | Op | alu_ctrl | src | wb | reg_we | mem_we | zero_we | carry_we | jmp | jz |
|---|---|---|---|---|---|---|---|---|---|---|
| 0000 | ADD | 0 | 0 | 0 | 1 | 0 | 1 | 1 | 0 | 0 |
| 0001 | SUB | 1 | 0 | 0 | 1 | 0 | 1 | 1 | 0 | 0 |
| 0010 | AND | 2 | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 0 |
| 0011 | OR | 3 | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 0 |
| 0100 | XOR | 4 | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 0 |
| 0101 | NOT | 5 | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 0 |
| 0110 | LDI | 6 | 1 | 0 | 1 | 0 | 0 | 0 | 0 | 0 |
| 0111 | MOV | 6 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 |
| 1000 | LOAD | 6 | 1 | 1 | 1 | 0 | 0 | 0 | 0 | 0 |
| 1001 | STORE | 6 | 1 | 0 | 0 | 1 | 0 | 0 | 0 | 0 |

| opcode | Op | alu_ctrl | src | wb | reg_we | mem_we | zero_we | carry_we | jmp | jz |
|---|---|---|---|---|---|---|---|---|---|---|
| 1010 | JMP | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 |
| 1011 | JZ | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 |
| 1100 | NOP | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 1101 | NOP | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 1110 | NOP | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 1111 | NOP | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

*Table 20. instruction_decoder - truth table for all 16 opcodes (simulation)*

| ex_is_jmp | ex_is_jz | zero_flag | pc_sel | flush | Meaning |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | no jump |
| 0 | 0 | 1 | 0 | 0 | no jump |
| 0 | 1 | 0 | 0 | 0 | JZ not taken |
| 0 | 1 | 1 | 1 | 1 | JZ taken |
| 1 | 0 | 0 | 1 | 1 | JMP: always taken |
| 1 | 0 | 1 | 1 | 1 | JMP: always taken |
| 1 | 1 | 0 | 1 | 1 | JMP: always taken |
| 1 | 1 | 1 | 1 | 1 | JMP: always taken |

*Table 21. branch_unit - truth table (simulation)*

The combinations with `ex_is_jmp` = 1 and `ex_is_jz` = 1 cannot occur because the decoder never raises both; the table shows that the branch unit would still jump.

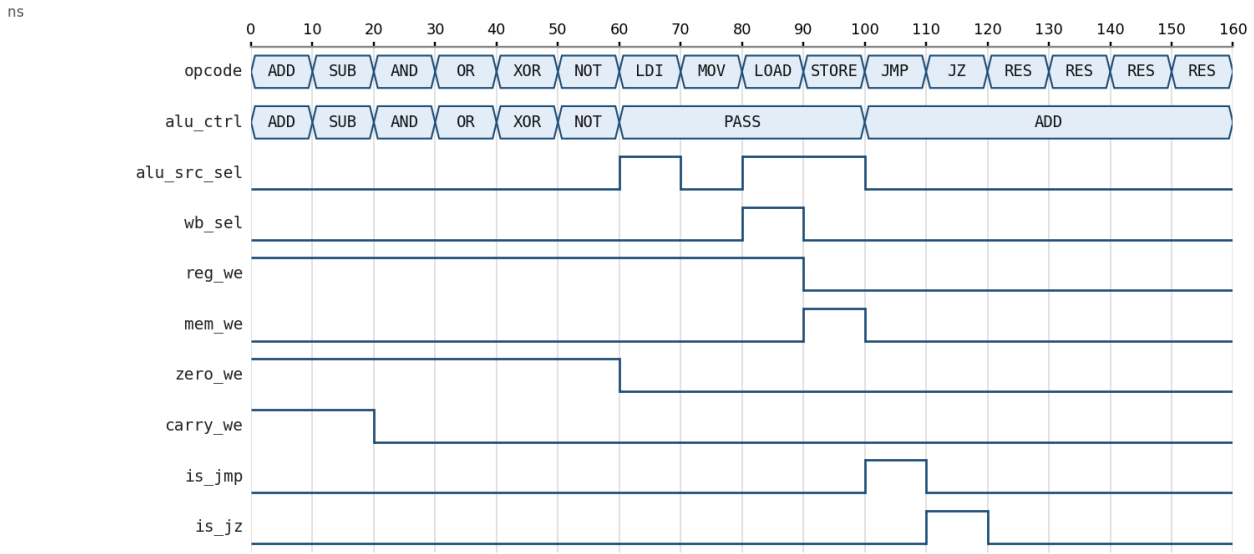



*Figure 16: Waveform of the instruction decoder: opcode swept from 0 to 15.*

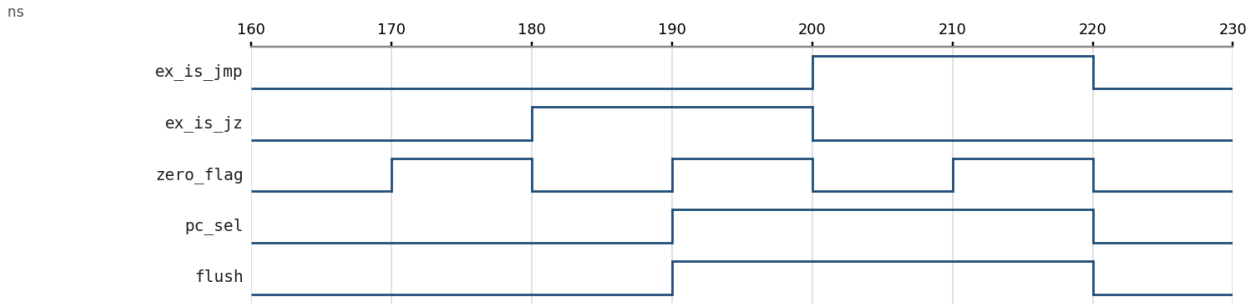



*Figure 17: Waveform of the branch unit.*

Waveform: the opcode is swept through all 16 values, 10 ns each. Arithmetic opcodes raise `reg_we` and `zero_we`, and only ADD and SUB also raise `carry_we`. LDI, LOAD and STORE use the PASS code (6) with `alu_src_sel` = 1; LOAD alone sets `wb_sel`, and STORE alone sets `mem_we` and does not set `reg_we`. JMP and JZ only raise `is_jmp` / `is_jz`. From opcode 1100 on, all outputs are 0. In the second figure, `pc_sel` and `flush` rise together exactly when `ex_is_jmp` = 1, or `ex_is_jz` = 1 with `zero_flag` = 1.

```
// ======================================================================
// FILE: control_unit.v
// Hardwired control unit for the 3-stage pipeline.
//
// Interface (as in the project plan):
//   opcode, flags, alu_ctrl, reg_we, mux selects, PC controls
//
// This file contains 3 modules:
//   1. instruction_decoder : DECODE stage - turns the opcode into
//                            control signals (pure logic, no clock)
//   2. branch_unit         : EXECUTE stage - decides if a jump is taken,
//                            and if so asks the pipeline to flush
//   3. control_unit        : joins the two modules above
//
// CONNECTIONS
//   Calls       : only modules inside this same file:
//                 instruction_decoder, branch_unit
//   Called by   : cpu_top.v - instance CU
//                 tb_control_unit.v - instance DUT
// ======================================================================


// ----------------------------------------------------------------------
// MODULE 1: instruction_decoder  (used in the DECODE stage)
// Turns ON only the signals an instruction needs.
// ----------------------------------------------------------------------
module instruction_decoder (
    input      [3:0] opcode,      // instruction bits [11:8]
    output reg [2:0] alu_ctrl,    // ALU operation
    output reg       alu_src_sel, // operand MUX: 0 = Rs, 1 = immediate
    output reg       wb_sel,      // write-back MUX: 0 = ALU, 1 = memory
    output reg       reg_we,      // 1 = write register Rd
    output reg       mem_we,      // 1 = write data memory
    output reg       zero_we,     // 1 = update Zero flag
    output reg       carry_we,    // 1 = update Carry flag
    output reg       is_jmp,      // 1 = this is JMP
    output reg       is_jz        // 1 = this is JZ
);
    // ---- opcodes (project plan, Section 4) ----
    localparam ADD   = 4'b0000;
    localparam SUB   = 4'b0001;
    localparam AND   = 4'b0010;
    localparam OR    = 4'b0011;
    localparam XOR   = 4'b0100;
    localparam NOT   = 4'b0101;
```

```
    localparam LDI   = 4'b0110;
    localparam MOV   = 4'b0111;
    localparam LOAD  = 4'b1000;
    localparam STORE = 4'b1001;
    localparam JMP   = 4'b1010;
    localparam JZ    = 4'b1011;
    // 1100 - 1111 are reserved: they do nothing (NOP)

    // ---- ALU control codes (must match alu.v) ----
    localparam ALU_ADD  = 3'd0;
    localparam ALU_SUB  = 3'd1;
    localparam ALU_AND  = 3'd2;
    localparam ALU_OR   = 3'd3;
    localparam ALU_XOR  = 3'd4;
    localparam ALU_NOT  = 3'd5;
    localparam ALU_PASS = 3'd6;

    always @(*) begin
        // ---- Step 1: default = do nothing ----
        alu_ctrl    = ALU_ADD;
        alu_src_sel = 1'b0;
        wb_sel      = 1'b0;
        reg_we      = 1'b0;
        mem_we      = 1'b0;
        zero_we     = 1'b0;
        carry_we    = 1'b0;
        is_jmp      = 1'b0;
        is_jz       = 1'b0;

        // ---- Step 2: turn on what the opcode needs ----
        case (opcode)
            ADD: begin                    // Rd = Rd + Rs
                alu_ctrl = ALU_ADD;
                reg_we   = 1'b1;
                zero_we  = 1'b1;
                carry_we = 1'b1;
            end
            SUB: begin                    // Rd = Rd - Rs
                alu_ctrl = ALU_SUB;
                reg_we   = 1'b1;
                zero_we  = 1'b1;
                carry_we = 1'b1;
            end
            AND: begin                    // Rd = Rd AND Rs
                alu_ctrl = ALU_AND;
                reg_we   = 1'b1;
                zero_we  = 1'b1;
            end
            OR: begin                     // Rd = Rd OR Rs
                alu_ctrl = ALU_OR;
                reg_we   = 1'b1;
                zero_we  = 1'b1;
            end
            XOR: begin                    // Rd = Rd XOR Rs
                alu_ctrl = ALU_XOR;
                reg_we   = 1'b1;
                zero_we  = 1'b1;
            end
            NOT: begin                    // Rd = NOT Rd
                alu_ctrl = ALU_NOT;
                reg_we   = 1'b1;
                zero_we  = 1'b1;
            end
            LDI: begin                    // Rd = immediate
                alu_ctrl    = ALU_PASS;   // ALU passes operand B
                alu_src_sel = 1'b1;       // operand B = immediate
                reg_we      = 1'b1;
            end
```

```
            MOV: begin                    // Rd = Rs
                alu_ctrl = ALU_PASS;      // ALU passes operand B = Rs
                reg_we   = 1'b1;
            end
            LOAD: begin                   // Rd = data_memory[addr]
                alu_ctrl    = ALU_PASS;   // ALU passes addr to memory
                alu_src_sel = 1'b1;
                wb_sel      = 1'b1;       // write memory data into Rd
                reg_we      = 1'b1;
            end
            STORE: begin                  // data_memory[addr] = Rd
                alu_ctrl    = ALU_PASS;   // ALU passes addr to memory
                alu_src_sel = 1'b1;
                mem_we      = 1'b1;
            end
            JMP: begin                    // PC = addr
                is_jmp = 1'b1;
            end
            JZ: begin                     // if Zero = 1, PC = addr
                is_jz = 1'b1;
            end
            default: begin
                // reserved opcodes 1100-1111: do nothing
            end
        endcase
    end
endmodule


// ---------------------------------------------------------------------
// MODULE 2: branch_unit  (used in the EXECUTE stage)
// pc_sel = 1 means "jump": the PC loads the jump address, and the two
// younger instructions already in IF and ID are flushed (made NOP).
// ---------------------------------------------------------------------
module branch_unit (
    input      is_jmp,        // instruction in EX is JMP
    input      is_jz,         // instruction in EX is JZ
    input      zero_flag,     // saved Zero flag
    output reg pc_sel,        // PC MUX: 0 = PC + 1, 1 = jump address
    output     flush          // 1 = empty the IF/ID and ID/EX registers
);
    always @(*) begin
        if (is_jmp)
            pc_sel = 1'b1;                // JMP always jumps
        else if (is_jz && zero_flag)
            pc_sel = 1'b1;                // JZ jumps only when Zero = 1
        else
            pc_sel = 1'b0;                // otherwise: next address
    end

    assign flush = pc_sel;                // jump taken -> flush
endmodule


// ---------------------------------------------------------------------
// MODULE 3: control_unit (instruction_decoder + branch_unit)
// The decoder works on the instruction in DECODE; the branch unit
// works on the instruction in EXECUTE.
// ---------------------------------------------------------------------
module control_unit (
    // ---- DECODE stage ----
    input      [3:0] opcode,
    output     [2:0] alu_ctrl,
    output           alu_src_sel,
    output           wb_sel,
    output           reg_we,
    output           mem_we,
    output           zero_we,
```

```
    output          carry_we,
    output          is_jmp,
    output          is_jz,
    // ---- EXECUTE stage ----
    input           ex_is_jmp,
    input           ex_is_jz,
    input           zero_flag,
    output          pc_sel,
    output          flush
);
    // CALLS: module instruction_decoder (MODULE 1 in this same file, control_unit.v)
    instruction_decoder DECODER (
        .opcode      (opcode),
        .alu_ctrl    (alu_ctrl),
        .alu_src_sel (alu_src_sel),
        .wb_sel      (wb_sel),
        .reg_we      (reg_we),
        .mem_we      (mem_we),
        .zero_we     (zero_we),
        .carry_we    (carry_we),
        .is_jmp      (is_jmp),
        .is_jz       (is_jz)
    );

    // CALLS: module branch_unit (MODULE 2 in this same file, control_unit.v)
    branch_unit BRANCH (
        .is_jmp    (ex_is_jmp),
        .is_jz     (ex_is_jz),
        .zero_flag (zero_flag),
        .pc_sel    (pc_sel),
        .flush     (flush)
    );
endmodule
```

***Source code: control_unit.v***

## 5.6. pipeline_register - ID/EX Pipeline Register

**Function:** this register separates the ID and EX stages. At every clock edge it copies everything the EX stage will need: the nine control bits, the register numbers Rd and Rs, the immediate, the instruction address, the valid bit and the complete instruction word (the last three are used for tracing and for the debug outputs). If `rst` or `flush` is 1 it loads a **bubble**: all control bits 0, all fields 0, valid 0, instruction C00. A bubble in EX writes no register, no memory word and no flag, and cannot cause a jump.

| Ports | Direction | Width | Description |
|---|---|---|---|
| clk, rst, flush | input | 1 each | clock, synchronous reset, flush |
| alu_ctrl_in / alu_ctrl | input / output | 3 | ALU operation code |
| alu_src_sel, wb_sel, reg_we, mem_we, zero_we, carry_we, is_jmp, is_jz (with suffix _in on the input side) | input / output | 1 each | decoded control bits |
| rd_in, rs_in / rd, rs | input / output | 2 each | register numbers |
| imm_in / imm | input / output | 4 | immediate, address or jump target |
| pc_in / pc | input / output | 4 | instruction address (tracing) |
| valid_in / valid | input / output | 1 | real instruction (1) or bubble (0) |
| instr_in / instr | input / output | 12 | instruction word (debug output instr_out) |

*Table 22. pipeline_register - port list (grouped)*

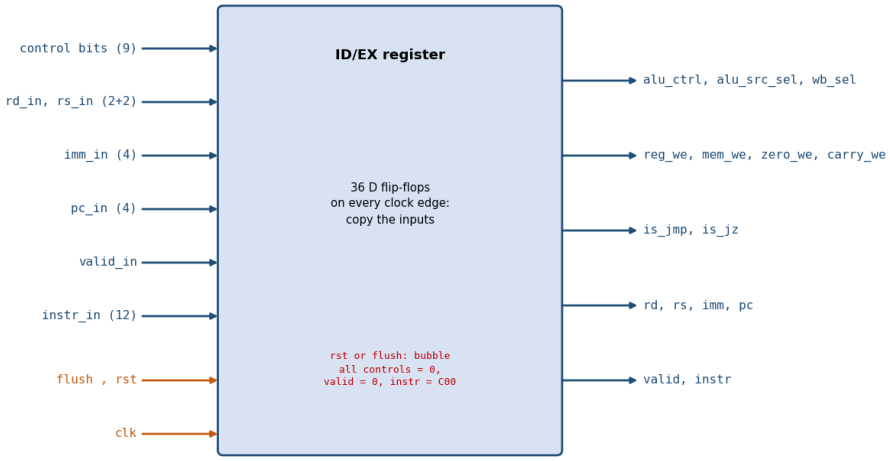



*Figure 18: Block diagram of pipeline_register.*

| rst | flush | alu_ctrl | reg_we | zero_we | carry_we | rd | rs | pc | valid | instr |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 1 | 1 | 1 | 1 | 2 | 2 | 1 | 060 |
| 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | C00 |
| 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | C00 |
| 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | C00 |

*Table 23. pipeline_register - result of one clock edge (input: ADD R1,R2 at address 2)*

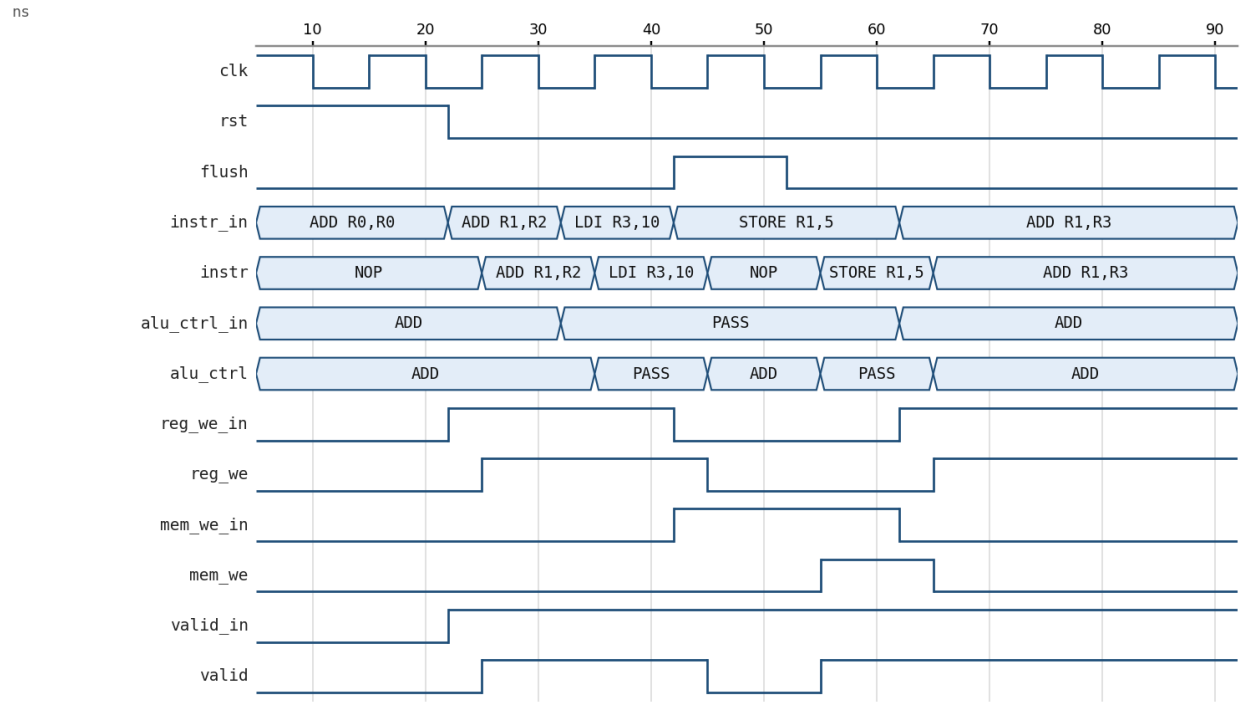



*Figure 19: Waveform of pipeline_register.*

Waveform: the input instructions are ADD R1,R2 (060), LDI R3,10 (6CA), STORE R1,5 (945) and ADD R1,R3 (070). The first two appear at the output one clock edge later with their control bits (ADD: `reg_we` = 1, ALU code 0; LDI: ALU code 6). At 45 ns `flush` is 1 while STORE is at the input, so the output at 47 ns is the bubble: instruction C00, `valid` = 0 and every write enable 0. The STORE is offered again after the flush ends and appears at the output with `mem_we` = 1 at 55 ns; ADD R1,R3 follows at 65 ns.

```
// ============================================================================
// FILE: pipeline_register.v
// The ID/EX PIPELINE REGISTER: it sits between the DECODE stage and
// the EXECUTE stage.
//
// At every clock it copies everything the EXECUTE stage will need:
//   - the control signals made by the decoder
//   - the instruction fields: Rd, Rs, imm
//   - the instruction address and a valid bit (for tracing)
//
// flush = 1 (taken jump) or rst = 1 loads a "do nothing" instruction:
// all control signals 0, so the EXECUTE stage changes nothing.
//
// CONNECTIONS
//   Calls       : nothing (no other modules)
//   Called by   : cpu_top.v - instance ID_EX
// ============================================================================
module pipeline_register (
    input             clk,
    input             rst,
    input             flush,
    // ---- inputs from the DECODE stage ----
    input      [2:0]  alu_ctrl_in,
    input             alu_src_sel_in,
    input             wb_sel_in,
    input             reg_we_in,
    input             mem_we_in,
    input             zero_we_in,
    input             carry_we_in,
    input             is_jmp_in,
    input             is_jz_in,
```

```
    input      [1:0]  rd_in,
    input      [1:0]  rs_in,
    input      [3:0]  imm_in,
    input      [3:0]  pc_in,
    input             valid_in,
    input      [11:0] instr_in,
    // ---- outputs to the EXECUTE stage ----
    output reg [2:0]  alu_ctrl,
    output reg        alu_src_sel,
    output reg        wb_sel,
    output reg        reg_we,
    output reg        mem_we,
    output reg        zero_we,
    output reg        carry_we,
    output reg        is_jmp,
    output reg        is_jz,
    output reg [1:0]  rd,
    output reg [1:0]  rs,
    output reg [3:0]  imm,
    output reg [3:0]  pc,
    output reg        valid,
    output reg [11:0] instr
);
    always @(posedge clk) begin
        if (rst || flush) begin
            // ---- bubble: nothing will be written in EXECUTE ----
            alu_ctrl    <= 3'd0;
            alu_src_sel <= 1'b0;
            wb_sel      <= 1'b0;
            reg_we      <= 1'b0;
            mem_we      <= 1'b0;
            zero_we     <= 1'b0;
            carry_we    <= 1'b0;
            is_jmp      <= 1'b0;
            is_jz       <= 1'b0;
            rd          <= 2'd0;
            rs          <= 2'd0;
            imm         <= 4'd0;
            pc          <= 4'd0;
            valid       <= 1'b0;
            instr       <= 12'hC00;
        end
        else begin
            // ---- pass the decoded instruction on to EXECUTE ----
            alu_ctrl    <= alu_ctrl_in;
            alu_src_sel <= alu_src_sel_in;
            wb_sel      <= wb_sel_in;
            reg_we      <= reg_we_in;
            mem_we      <= mem_we_in;
            zero_we     <= zero_we_in;
            carry_we    <= carry_we_in;
            is_jmp      <= is_jmp_in;
            is_jz       <= is_jz_in;
            rd          <= rd_in;
            rs          <= rs_in;
            imm         <= imm_in;
            pc          <= pc_in;
            valid       <= valid_in;
            instr       <= instr_in;
        end
    end
endmodule
```

***Source code: pipeline_register.v***

## 5.7. register_file - Register File

**Function:** four 4-bit general-purpose registers R0 to R3 with two combinational read ports (`qa` for address `ra`, `qb` for address `rb`) and one write port that is written at the clock edge when `we` = 1. A reset clears all four registers and has priority over the write. In `cpu_top` the first read address and the write address are both the destination register Rd of the instruction in EX, so an instruction such as ADD reads its first operand and writes its result into the same register.

| Port | Direction | Description |
|---|---|---|
| clk, rst | input | clock and synchronous reset (all registers <- 0) |
| ra, rb [1:0] | input | read addresses of port A and port B |
| wa [1:0], wd [3:0], we | input | write address, write data, write enable |
| qa, qb [3:0] | output | read data (combinational) |

*Table 24. register_file - port list*

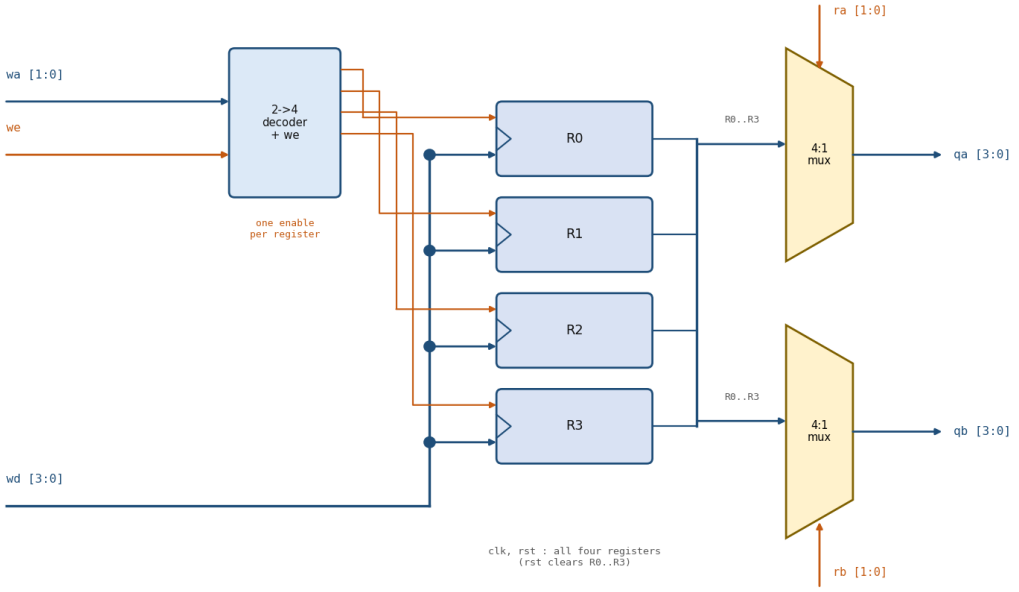



*Figure 20: Block diagram of register_file.*

| Operation | R0 | R1 | R2 | R3 |
|---|---|---|---|---|
| Reset (rst = 1) | 0000 | 0000 | 0000 | 0000 |
| Write R1 <- 0101 | 0000 | 0101 | 0000 | 0000 |
| Write R2 <- 0011 | 0000 | 0101 | 0011 | 0000 |
| Write R1 <- 1111 (overwrite) | 0000 | 1111 | 0011 | 0000 |
| we = 0, wa = R2 (ignored) | 0000 | 1111 | 0011 | 0000 |
| rst = 1 and we = 1 (reset wins) | 0000 | 0000 | 0000 | 0000 |
| Write R0 <- 1001 | 1001 | 0000 | 0000 | 0000 |

*Table 25. register_file - register contents after each clock edge (simulation)*

| ra | rb | qa | qb |
|---|---|---|---|
| R0 | R1 | 1001 | 0110 |
| R1 | R0 | 0110 | 1001 |
| R1 | R1 | 0110 | 0110 |

*Table 26. register_file - read ports (R0 = 1001, R1 = 0110)*

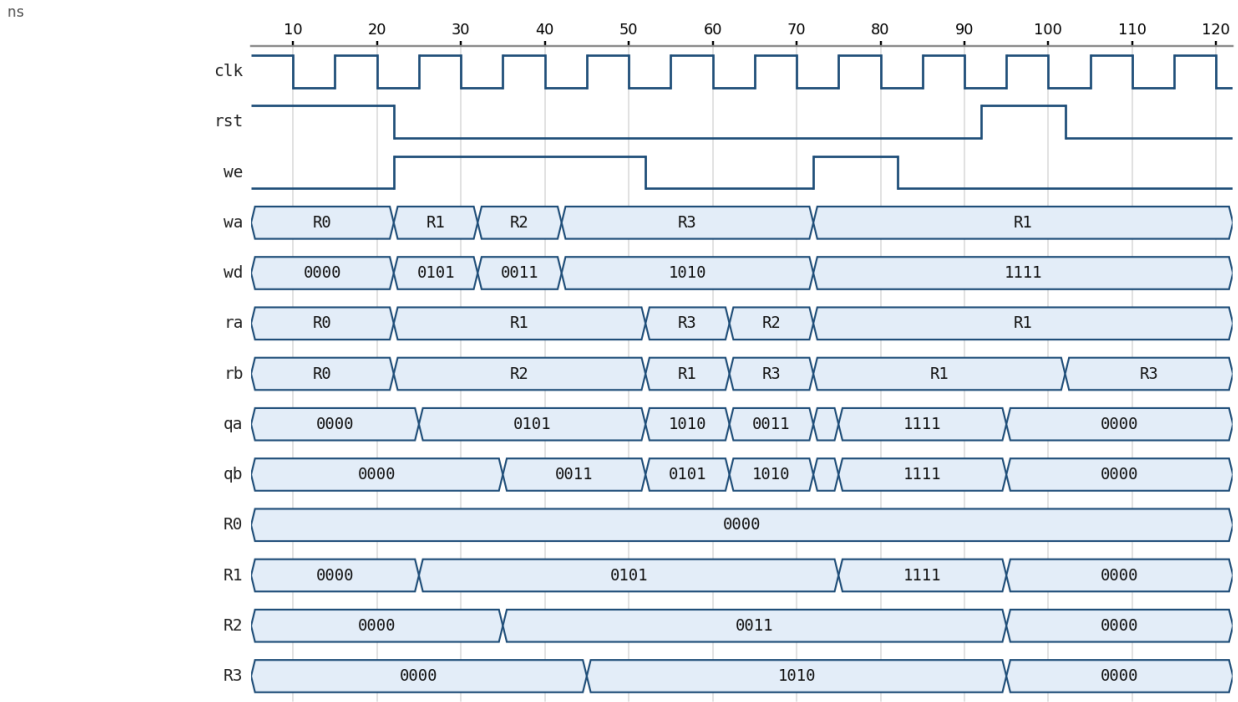



*Figure 21: Waveform of register_file.*

Waveform: at 23 ns the write of 0101 into R1 is requested while `ra` = R1; the output `qa` is still 0000 until the clock edge at 25 ns and is 0101 afterwards, showing that reads are combinational and writes are clocked. R2 and R3 are written at 35 ns and 45 ns (0011 and 1010). With `we` = 0, reading R3/R1 gives 1010/0101 and reading R2/R3 gives 0011/1010. At 75 ns R1 is overwritten by 1111, and the reset at 95 ns clears all registers (`qa` = `qb` = 0000).

```
// ======================================================================
// FILE: register_file.v
// 4 general-purpose registers (R0-R3), each 4 bits wide.
//
// Interface (as in the project plan):
//   clk, rst, ra, rb, wa, wd, we, qa, qb
//
//   ra, qa : read port A  (address in, value out)  - no clock needed
//   rb, qb : read port B  (address in, value out)  - no clock needed
//   wa, wd : write address and write data          - on clock edge
//   we     : write enable (1 = write)
//
// Register numbers: R0 = 00, R1 = 01, R2 = 10, R3 = 11
//
// CONNECTIONS
//   Calls       : nothing (no other modules)
//   Called by   : cpu_top.v - instance RF
//                 tb_register_file.v - instance DUT
//                 tb_cpu.v - reads CPU.RF.r0 ... CPU.RF.r3
// ======================================================================
module register_file (
    input            clk,
    input            rst,
    input      [1:0] ra,          // read address A
    input      [1:0] rb,          // read address B
    input      [1:0] wa,          // write address
    input      [3:0] wd,          // write data
    input            we,          // write enable
    output reg [3:0] qa,          // read data A
    output reg [3:0] qb           // read data B
);
```

```
    // the four registers
    reg [3:0] r0, r1, r2, r3;

    // ---------------- WRITE (clocked) ----------------
    always @(posedge clk) begin
        if (rst) begin
            r0 <= 4'b0000;        // reset clears all registers
            r1 <= 4'b0000;
            r2 <= 4'b0000;
            r3 <= 4'b0000;
        end
        else if (we) begin
            case (wa)
                2'd0: r0 <= wd;
                2'd1: r1 <= wd;
                2'd2: r2 <= wd;
                2'd3: r3 <= wd;
            endcase
        end
    end

    // ---------------- READ PORT A (combinational) ----------------
    always @(*) begin
        case (ra)
            2'd0: qa = r0;
            2'd1: qa = r1;
            2'd2: qa = r2;
            2'd3: qa = r3;
        endcase
    end

    // ---------------- READ PORT B (combinational) ----------------
    always @(*) begin
        case (rb)
            2'd0: qb = r0;
            2'd1: qb = r1;
            2'd2: qb = r2;
            2'd3: qb = r3;
        endcase
    end
endmodule
```

***Source code: register_file.v***

## 5.8. alu - Arithmetic Logic Unit

**Function:** the ALU performs every calculation of the processor. It is built from three small modules in `alu.v`: **adder_subtractor** (ADD and SUB with carry), **logic_unit** (AND, OR, XOR, NOT and PASS) and **zero_detector** (1 when the result is 0000), joined by the top module `alu`. `alu_ctrl` selects the operation: for codes 0 and 1 the adder result and carry are used; for all other codes the logic result is used and the carry is 0. The PASS operation (code 6) copies operand b to the output; it is used by LDI (b = immediate), MOV (b = Rs) and by LOAD and STORE, where the address in the immediate field passes through the ALU to the data memory. Code 7 is unused and returns 0000.

| Port | Direction | Description |
|---|---|---|
| a [3:0] | input | operand A (register Rd) |
| b [3:0] | input | operand B (register Rs or immediate) |
| alu_ctrl [2:0] | input | 0 ADD, 1 SUB, 2 AND, 3 OR, 4 XOR, 5 NOT (of a), 6 PASS (b), 7 unused |
| result [3:0] | output | result |
| carry | output | ADD: carry-out; SUB: 1 = no borrow; logic: 0 |
| zero | output | 1 when result = 0000 |

*Table 27. alu - port list*

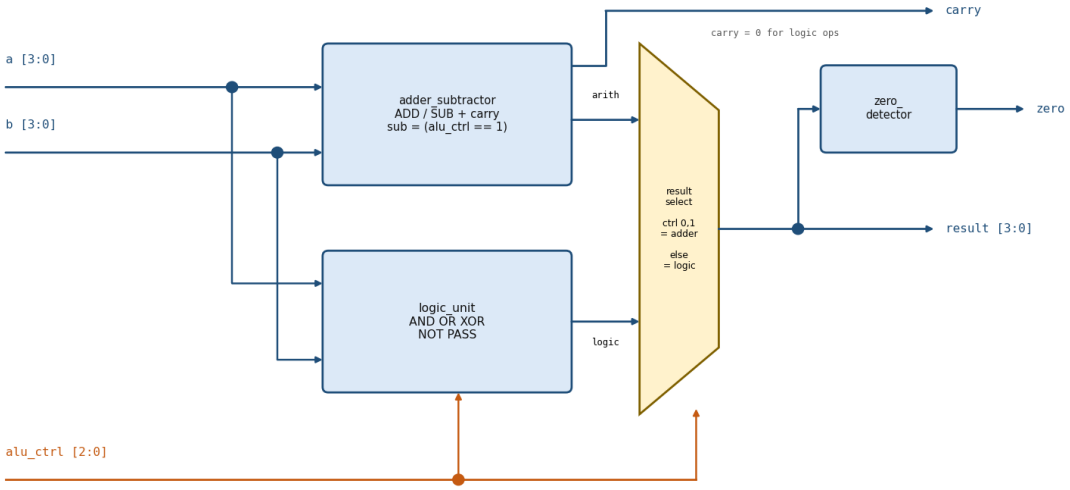



*Figure 22: Block diagram of alu.*

| alu_ctrl | Operation | a | b | result | carry | zero |
|---|---|---|---|---|---|---|
| 0 | ADD | 0101 | 0011 | 1000 | 0 | 0 |
| 0 | ADD | 1111 | 0001 | 0000 | 1 | 1 |
| 0 | ADD | 1000 | 1000 | 0000 | 1 | 1 |
| 0 | ADD | 0000 | 0000 | 0000 | 0 | 1 |
| 1 | SUB | 0101 | 0011 | 0010 | 1 | 0 |
| 1 | SUB | 0011 | 0101 | 1110 | 0 | 0 |
| 1 | SUB | 0111 | 0111 | 0000 | 1 | 1 |
| 1 | SUB | 0000 | 0001 | 1111 | 0 | 0 |
| 2 | AND | 1100 | 1010 | 1000 | 0 | 0 |
| 2 | AND | 1010 | 0101 | 0000 | 0 | 1 |
| 3 | OR | 1100 | 1010 | 1110 | 0 | 0 |
| 3 | OR | 0000 | 0000 | 0000 | 0 | 1 |

| alu_ctrl | Operation | a | b | result | carry | zero |
|---|---|---|---|---|---|---|
| 4 | XOR | 1100 | 1010 | 0110 | 0 | 0 |
| 4 | XOR | 0110 | 0110 | 0000 | 0 | 1 |
| 5 | NOT | 0101 | 1111 | 1010 | 0 | 0 |
| 5 | NOT | 1111 | 0000 | 0000 | 0 | 1 |
| 6 | PASS | 1111 | 1001 | 1001 | 0 | 0 |
| 6 | PASS | 1111 | 0000 | 0000 | 0 | 1 |
| 7 | reserved | 1010 | 0101 | 0000 | 0 | 1 |

*Table 28. alu - truth table (simulation, a and b in binary)*

Selected rows explain the flag rules: 1111 + 0001 gives 0000 with carry 1 and zero 1; 1000 + 1000 also gives carry 1 and zero 1; 0011 - 0101 gives 1110 with carry 0 (a borrow); 0111 - 0111 gives 0000 with carry 1 (no borrow). The logic operations always report carry 0, and the reserved code 7 returns 0000 with zero 1.

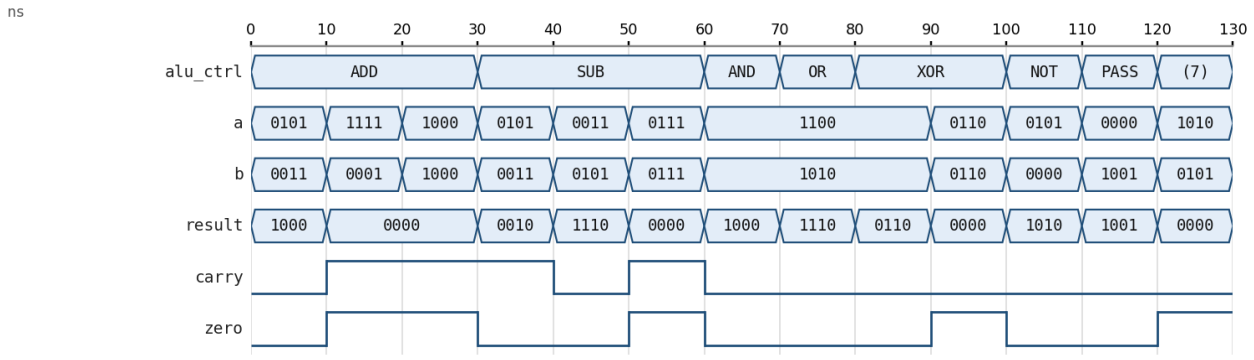



*Figure 23: Waveform of alu.*

Waveform: the first 130 ns step through every operation. For ADD, 5 + 3 = 1000 with no flags; 15 + 1 = 0000 with carry and zero; 8 + 8 = 0000 with carry and zero. For SUB, 5 - 3 = 0010 (carry 1), 3 - 5 = 1110 (carry 0) and 7 - 7 = 0000 (carry 1, zero 1). The logic operations follow with carry 0, then NOT (0101 gives 1010), PASS (result = b = 1001) and the reserved code 7 (result 0000, zero 1).

```
// ====================================================================
// FILE: alu.v
// The ALU (Arithmetic Logic Unit) does all the calculations.
//
// Interface (as in the project plan):
//   a[3:0], b[3:0], alu_ctrl, result[3:0], carry, zero
//
// This file contains 4 small modules:
//   1. adder_subtractor : does ADD and SUB, gives the carry
//   2. logic_unit       : does AND, OR, XOR, NOT and PASS (copy b)
//   3. zero_detector    : checks if the result is 0000
//   4. alu              : the main ALU, it joins the 3 parts above
//
// ALU control codes (alu_ctrl):
//   0 = ADD    1 = SUB    2 = AND    3 = OR
//   4 = XOR    5 = NOT    6 = PASS (result = b; used by LDI, MOV,
//                                   LOAD and STORE)
//
// CARRY CONVENTION (same in ALU, flag register and testbenches):
//   ADD : carry = 1 when the sum is bigger than 15 (carry-out)
//   SUB : carry = 1 when a >= b, i.e. NO BORROW
//         carry = 0 when a <  b, i.e. a borrow happened
//
// CONNECTIONS
//   Calls       : only modules inside this same file:
//                 adder_subtractor, logic_unit, zero_detector
```

```
//   Called by   : cpu_top.v - instance ALU
//                 tb_alu.v - instance DUT
// ====================================================================


// --------------------------------------------------------------------
// MODULE 1: adder_subtractor
// --------------------------------------------------------------------
module adder_subtractor (
    input      [3:0] a,
    input      [3:0] b,
    input            do_sub,     // 0 = add, 1 = subtract
    output reg [3:0] result,
    output reg       carry
);
    reg [4:0] full_sum;          // 5 bits, so we can see the carry bit

    always @(*) begin
        full_sum = 5'b00000;
        if (do_sub == 1'b0) begin
            // ---- ADD ----
            full_sum = a + b;            // e.g. 15 + 1 = 16 = 1_0000
            result   = full_sum[3:0];    // lower 4 bits are the answer
            carry    = full_sum[4];      // 5th bit is the carry
        end
        else begin
            // ---- SUB ----
            result = a - b;              // e.g. 5 - 3 = 2
            if (a >= b)
                carry = 1'b1;            // no borrow
            else
                carry = 1'b0;            // borrow happened
        end
    end
endmodule


// --------------------------------------------------------------------
// MODULE 2: logic_unit
// --------------------------------------------------------------------
module logic_unit (
    input      [3:0] a,
    input      [3:0] b,
    input      [2:0] alu_ctrl,
    output reg [3:0] result
);
    always @(*) begin
        case (alu_ctrl)
            3'd2:    result = a & b;     // AND
            3'd3:    result = a | b;     // OR
            3'd4:    result = a ^ b;     // XOR
            3'd5:    result = ~a;        // NOT (only uses a)
            3'd6:    result = b;         // PASS: copy b
            default: result = 4'b0000;
        endcase
    end
endmodule


// --------------------------------------------------------------------
// MODULE 3: zero_detector
// --------------------------------------------------------------------
module zero_detector (
    input  [3:0] value,
    output       zero
);
    assign zero = (value == 4'b0000) ? 1'b1 : 1'b0;
endmodule
```

```
// ------------------------------------------------------------------------
// MODULE 4: alu (main ALU)
// ------------------------------------------------------------------------
module alu (
    input      [3:0] a,          // operand A (register Rd)
    input      [3:0] b,          // operand B (register Rs or immediate)
    input      [2:0] alu_ctrl,   // which operation to do
    output reg [3:0] result,     // final answer
    output reg       carry,      // carry / no-borrow
    output           zero        // 1 when result = 0000
);
    wire [3:0] arith_result;     // answer from adder_subtractor
    wire       arith_carry;      // carry  from adder_subtractor
    wire [3:0] logic_result;     // answer from logic_unit

    wire do_sub = (alu_ctrl == 3'd1);    // alu_ctrl 1 means SUB

    // CALLS: module adder_subtractor (MODULE 1 in this same file, alu.v)
    adder_subtractor ADDSUB (
        .a(a), .b(b), .do_sub(do_sub),
        .result(arith_result), .carry(arith_carry)
    );

    // CALLS: module logic_unit (MODULE 2 in this same file, alu.v)
    logic_unit LOGIC (
        .a(a), .b(b), .alu_ctrl(alu_ctrl),
        .result(logic_result)
    );

    // choose the final answer
    always @(*) begin
        if (alu_ctrl == 3'd0 || alu_ctrl == 3'd1) begin
            result = arith_result;       // ADD or SUB
            carry  = arith_carry;
        end
        else begin
            result = logic_result;       // AND, OR, XOR, NOT, PASS
            carry  = 1'b0;               // logic ops have no carry
        end
    end

    // CALLS: module zero_detector (MODULE 3 in this same file, alu.v)
    zero_detector ZERO (
        .value(result),
        .zero(zero)
    );
endmodule
```

***Source code: alu.v***

## 5.9. flag_register - Zero and Carry Flags

**Function:** the flag register stores the Zero and Carry flags so that a later instruction (JZ) can use them and so that they can be observed. Each flag has its own write enable: ADD and SUB enable both, AND, OR, XOR and NOT enable only Zero, and all other instructions enable neither. A flag therefore keeps the value written by the last instruction that was allowed to change it. A synchronous reset clears both flags.

| Port | Direction | Description |
|---|---|---|
| clk, rst | input | clock, synchronous reset (both flags <- 0) |
| zero_we, carry_we | input | update enable of Zero / Carry |
| zero_in, carry_in | input | new values from the ALU |
| zero_flag, carry_flag | output | saved flags |

*Table 29. flag_register - port list*

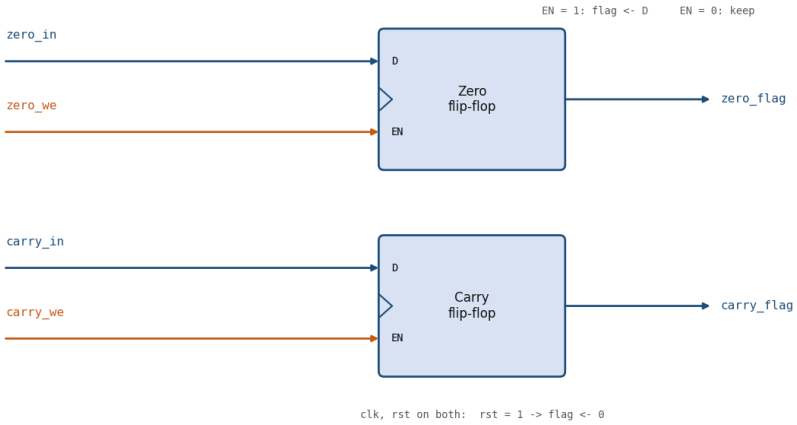



*Figure 24: Block diagram of flag_register.*

| Start Z C | zero_we | carry_we | zero_in | carry_in | Result Z C |
|---|---|---|---|---|---|
| 1 1 | 0 | 0 | 0 | 0 | 1 1 |
| 1 1 | 0 | 1 | 0 | 0 | 1 0 |
| 1 1 | 1 | 0 | 0 | 0 | 0 1 |
| 1 1 | 1 | 1 | 0 | 0 | 0 0 |
| 0 0 | 0 | 0 | 1 | 1 | 0 0 |
| 0 0 | 0 | 1 | 1 | 1 | 0 1 |
| 0 0 | 1 | 0 | 1 | 1 | 1 0 |
| 0 0 | 1 | 1 | 1 | 1 | 1 1 |
| 1 1 | rst = 1 | | | | 0 0 |

*Table 30. flag_register - result of one clock edge (simulation)*

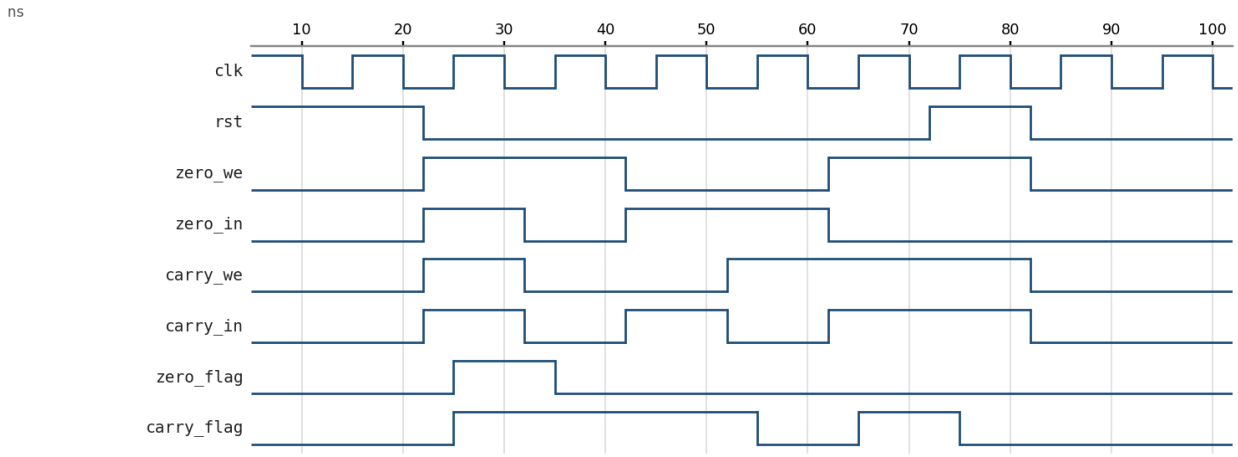



*Figure 25: Waveform of flag_register.*

Waveform: at 25 ns both enables are 1 with both inputs 1, so Z = C = 1. At 35 ns only `zero_we` is 1 and Z becomes 0 while C stays 1, like a logic instruction. At 45 ns no enable is set, and the inputs (1, 1) are ignored. At 55 ns only `carry_we` is 1 and C becomes 0; at 65 ns both are enabled with inputs Z = 0, C = 1. The reset at 75 ns clears both flags.

```
// ======================================================================
// FILE: flag_register.v
// Saves the Zero and Carry flags so a later instruction (JZ) can use
// them, and so they can be observed in the waveform.
//   zero  : last arithmetic/logic result was 0000
//   carry : carry-out from ADD, or NO-BORROW from SUB
// They have separate enables because AND/OR/XOR/NOT change only Zero,
// while ADD/SUB change both Zero and Carry.
//
// CONNECTIONS
//   Calls       : nothing (no other modules)
//   Called by   : cpu_top.v - instance FLAGS
// ======================================================================
module flag_register (
    input      clk,
    input      rst,
    input      zero_we,     // 1 = save new zero
    input      carry_we,    // 1 = save new carry
    input      zero_in,     // from the ALU
    input      carry_in,    // from the ALU
    output reg zero_flag,   // saved zero
    output reg carry_flag   // saved carry
);
    always @(posedge clk) begin
        if (rst) begin
            zero_flag  <= 1'b0;
            carry_flag <= 1'b0;
        end
        else begin
            if (zero_we)  zero_flag  <= zero_in;
            if (carry_we) carry_flag <= carry_in;
        end
    end
endmodule
```

***Source code: flag_register.v***

## 5.10. data_memory - Data RAM

**Function:** a 16 x 4-bit RAM used by LOAD and STORE. A write happens at the rising clock edge when `mem_we` = 1, at the address and with the data that are present at that edge. The read is asynchronous, so `read_data` always shows the word at `address`; a LOAD therefore needs no extra cycle. All words are initialised to 0000 so that no unknown value can be read. The memory is separate from the instruction memory (Harvard organisation).

| Port | Direction | Description |
|---|---|---|
| clk | input | clock for the write |
| address [3:0] | input | word address (ALU result) |
| write_data [3:0] | input | data to store (value of Rd) |
| mem_we | input | 1 = write at the clock edge |
| read_data [3:0] | output | word at address (combinational) |

*Table 31. data_memory - port list*

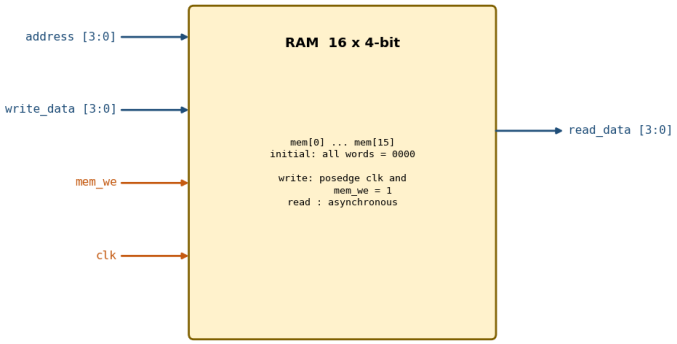



*Figure 26: Block diagram of data_memory.*

| Operation | mem_we | address | write_data | read_data | mem[3] afterwards |
|---|---|---|---|---|---|
| Store 0101 into mem[3] | 1 | 0011 | 0101 | 0101 | 0101 |
| Read mem[3] | 0 | 0011 | 0000 | 0101 | 0101 |
| mem_we = 0, write_data = 1111 (ignored) | 0 | 0011 | 1111 | 0101 | 0101 |
| Store 1111 into mem[3] | 1 | 0011 | 1111 | 1111 | 1111 |
| Read mem[4] (never written) | 0 | 0100 | 1111 | 0000 | 1111 |

*Table 32. data_memory - operations at the clock edge (simulation)*

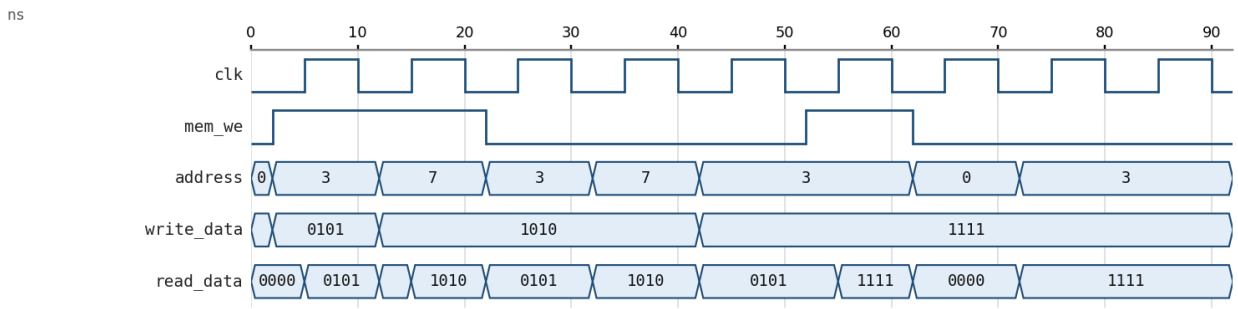



*Figure 27: Waveform of data_memory.*

Waveform: 0101 is stored at address 3 at 5 ns and 1010 at address 7 at 15 ns; `read_data` shows each value right after the edge. With `mem_we` = 0 the words are read back (0101, 1010). At 45 ns `write_data` is 1111 but `mem_we`

is 0, so mem[3] keeps 0101. At 55 ns the write is enabled and mem[3] becomes 1111. Address 0, never written, reads 0000.

```
// ======================================================================
// FILE: data_memory.v
// Data RAM: 16 words x 4 bits. Used by LOAD and STORE.
//
// Interface (as in the project plan):
//   clk, address, write_data, mem_we, read_data
//
// - Read  : any time (no clock needed)          -> LOAD
// - Write : on the clock edge when mem_we = 1    -> STORE
// It is separate from the instruction memory (Harvard organisation).
//
// CONNECTIONS
//   Calls       : nothing (no other modules)
//   Called by   : cpu_top.v - instance DMEM
//                 tb_cpu.v - reads and clears CPU.DMEM.mem
// ======================================================================
module data_memory (
    input        clk,
    input  [3:0] address,         // which word
    input  [3:0] write_data,      // data to write (STORE)
    input        mem_we,          // write enable: 1 = write
    output [3:0] read_data        // data read (LOAD)
);
    reg [3:0] mem [0:15];         // 16 words, 4 bits each
    integer i;

    // start with all words = 0 (no unknown X values)
    initial begin
        for (i = 0; i < 16; i = i + 1)
            mem[i] = 4'b0000;
    end

    // write on the clock edge
    always @(posedge clk) begin
        if (mem_we)
            mem[address] <= write_data;
    end

    // read any time
    assign read_data = mem[address];
endmodule
```

***Source code: data_memory.v***

## 5.11. cpu_top - Top-Level Module

**Function:** `cpu_top` instantiates all other modules and connects them as drawn in Figure 5; it contains no logic besides the adder that computes `pc + 1`. The three stages work simultaneously on three different instructions. The IF stage contains PC_MUX, PC and IMEM; the IF/ID register IR stores the fetched word. The ID stage splits the instruction and lets the decoder in CU generate the control bits, which are stored in ID_EX. The EX stage reads Rd and Rs from RF, selects operand B with OPERAND_MUX, calculates in ALU, updates FLAGS, accesses DMEM and writes the result back through WB_MUX. In the same stage, the branch unit in CU uses the saved Zero flag to produce `pc_sel` and `flush`. The interface of `cpu_top` is described in Section 4.1.

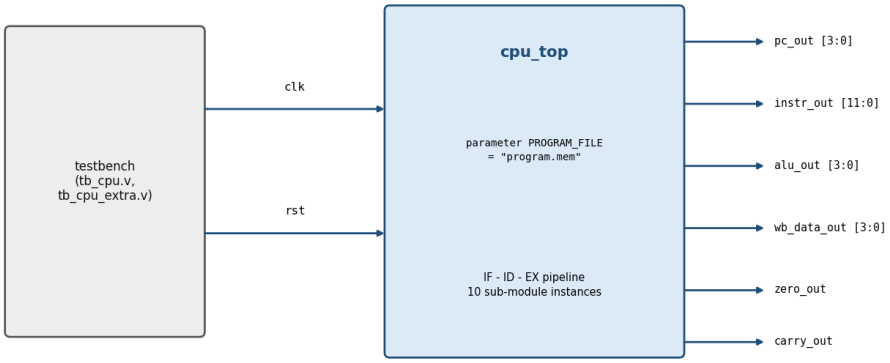



*Figure 28: Interface of cpu_top in its testbench.*

| Cycle | Addr | Instr | Assembly | R0 | R1 | R2 | R3 | Z | C |
|---|---|---|---|---|---|---|---|---|---|
| 3 | 0 | 645 | LDI R1,5 | 0000 | 0101 | 0000 | 0000 | 0 | 0 |
| 4 | 1 | 683 | LDI R2,3 | 0000 | 0101 | 0011 | 0000 | 0 | 0 |
| 5 | 2 | 060 | ADD R1,R2 | 0000 | 1000 | 0011 | 0000 | 0 | 0 |
| 6 | 3 | 6CA | LDI R3,10 | 0000 | 1000 | 0011 | 1010 | 0 | 0 |
| 7 | 4 | 470 | XOR R1,R3 | 0000 | 0010 | 0011 | 1010 | 0 | 0 |
| 8 | 5 | A05 | JMP 5 | 0000 | 0010 | 0011 | 1010 | 0 | 0 |

*Table 33. cpu_top - execution of program.mem (simulation, registers in binary)*

The table lists the state after each instruction has written its result. R1 = 0101 and R2 = 0011 are loaded, ADD gives R1 = 1000, LDI loads R3 = 1010 and XOR gives R1 = 1000 xor 1010 = 0010. The flags stay 0 because neither result is zero and no carry occurs. The final `JMP 5` keeps the program in a loop.

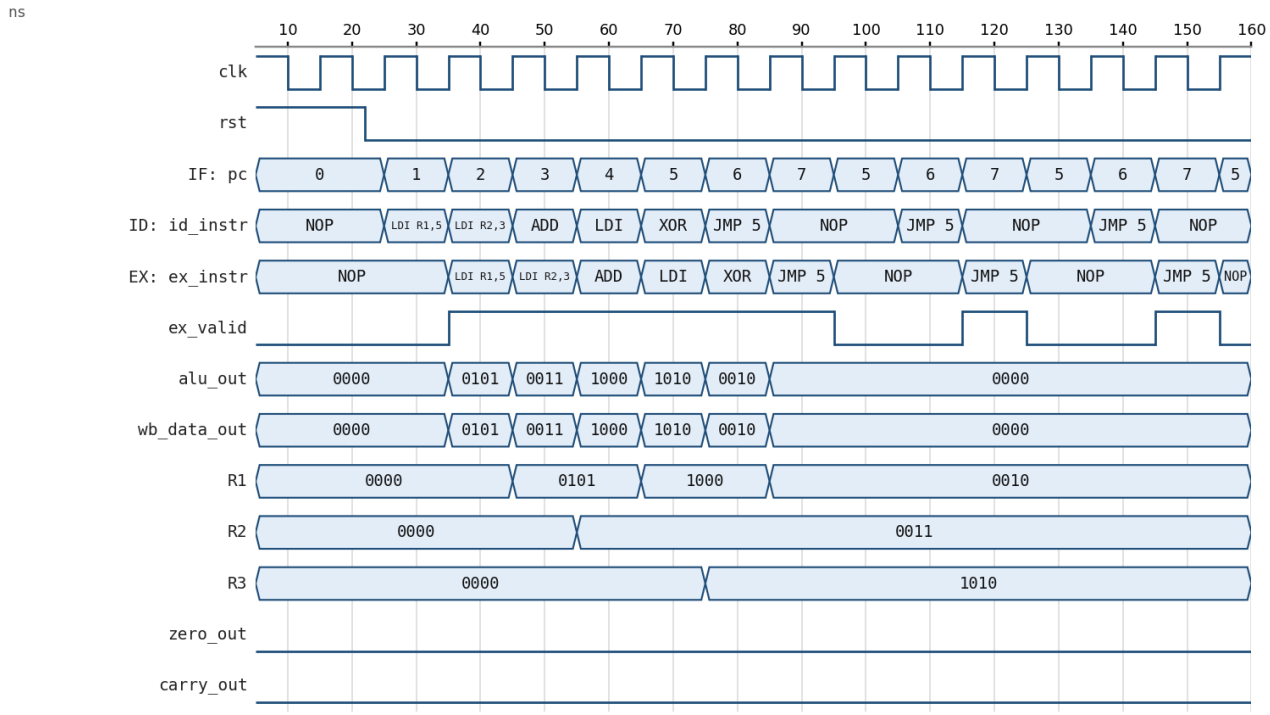



*Figure 29: Waveform of cpu_top running program.mem.*

Waveform: the first instruction (`LDI R1,5`) is fetched at 25 ns, is in ID during the next cycle and in EX from 35 ns; R1 changes to 0101 at 45 ns. The same pattern is repeated one cycle later for each following instruction: R2 = 0011 at 55 ns, R1 = 1000 (ADD) at 65 ns, R3 = 1010 at 75 ns and R1 = 0010 (XOR) at 85 ns. `alu_out` and `wb_data_out` show the value to be written during the EX cycle, one cycle before the register changes. After `JMP 5` enters EX the NOP bubbles appear in ID and EX (`ex_valid` = 0) and the fetch address returns to 5, so the loop repeats every three cycles.

```
// ======================================================================
// FILE: cpu_top.v
// The complete 4-bit RISC CPU with a 3-STAGE PIPELINE.
// This file only CONNECTS the blocks.
//
//   STAGE 1  IF (fetch)   : PC -> instruction memory -> IF/ID register (IR)
//   STAGE 2  ID (decode)  : control unit decodes -> ID/EX register
//   STAGE 3  EX (execute) : read registers -> operand MUX -> ALU
//                           -> data memory -> write-back MUX -> Rd,
//                           update flags, decide jumps
//
// All three stages work at the same time on three different
// instructions, so one instruction can finish every clock.
// A taken jump (decided in EX) flushes the 2 younger instructions.
//
// Instruction format (12 bits):
//   [11:8] opcode | [7:6] Rd | [5:4] Rs | [3:0] immediate / address
//
// CONNECTIONS
//   Calls       : mux.v, program_counter.v, instruction_memory.v,
//                 instruction_register.v, control_unit.v, pipeline_register.v,
//                 register_file.v, alu.v, flag_register.v, data_memory.v
//   Called by   : tb_cpu.v - instance CPU
// ======================================================================
module cpu_top #(
    parameter PROGRAM_FILE = "program.mem"   // program to run
)(
    input         clk,
    input         rst,
```

```
    // observable debug outputs (waveform viewer or FPGA LEDs)
    output [3:0]  pc_out,          // fetch address (PC)
    output [11:0] instr_out,       // instruction in the EXECUTE stage
    output [3:0]  alu_out,         // ALU result
    output [3:0]  wb_data_out,     // value written to the destination register
    output        zero_out,        // Zero flag
    output        carry_out        // Carry flag
);

    // wires shared between stages
    wire        pc_sel;            // from the branch unit (EX)
    wire        flush;             // from the branch unit (EX)
    wire [3:0]  ex_imm;            // jump address (EX)

    // ================================================================
    // STAGE 1: IF - FETCH
    // ================================================================
    wire [3:0]  pc;                // fetch address
    wire [3:0]  pc_plus_1 = pc + 4'd1;
    wire [3:0]  pc_next;
    wire [11:0] fetched;           // word out of instruction memory

    // PC MUX: next address = PC + 1, or the jump address from EX
    // CALLS: module mux2 from file mux.v
    mux2 PC_MUX (
        .d0  (pc_plus_1),
        .d1  (ex_imm),
        .sel (pc_sel),
        .y   (pc_next)
    );

    // CALLS: module program_counter from file program_counter.v
    program_counter PC (
        .clk     (clk),
        .rst     (rst),
        .pc_en   (1'b1),           // fetch a new instruction every clock
        .pc_next (pc_next),
        .pc_out  (pc)
    );

    // CALLS: module instruction_memory from file instruction_memory.v
    instruction_memory #(.PROGRAM_FILE(PROGRAM_FILE)) IMEM (
        .address     (pc),
        .instruction (fetched)
    );

    // ---------------- IF/ID pipeline register (the IR) ----------------
    wire [11:0] id_instr;          // instruction now in DECODE
    wire [3:0]  id_pc;
    wire        id_valid;

    // CALLS: module instruction_register from file instruction_register.v
    instruction_register IR (
        .clk             (clk),
        .rst             (rst),
        .ir_load         (1'b1),   // load every clock
        .flush           (flush),
        .instruction_in  (fetched),
        .pc_in           (pc),
        .instruction_out (id_instr),
        .pc_out          (id_pc),
        .valid_out       (id_valid)
    );

    // ================================================================
    // STAGE 2: ID - DECODE
    // ================================================================
    wire [3:0] id_opcode = id_instr[11:8];
```

```
    wire [1:0] id_rd     = id_instr[7:6];
    wire [1:0] id_rs     = id_instr[5:4];
    wire [3:0] id_imm    = id_instr[3:0];

    wire [2:0] id_alu_ctrl;
    wire       id_alu_src_sel, id_wb_sel, id_reg_we, id_mem_we;
    wire       id_zero_we, id_carry_we, id_is_jmp, id_is_jz;

    wire       ex_is_jmp, ex_is_jz;   // from ID/EX (below)
    wire       zero_flag, carry_flag; // from the flag register (EX)

    // CALLS: module control_unit from file control_unit.v
    control_unit CU (
        // decode part
        .opcode      (id_opcode),
        .alu_ctrl    (id_alu_ctrl),
        .alu_src_sel (id_alu_src_sel),
        .wb_sel      (id_wb_sel),
        .reg_we      (id_reg_we),
        .mem_we      (id_mem_we),
        .zero_we     (id_zero_we),
        .carry_we    (id_carry_we),
        .is_jmp      (id_is_jmp),
        .is_jz       (id_is_jz),
        // branch part (EX stage)
        .ex_is_jmp   (ex_is_jmp),
        .ex_is_jz    (ex_is_jz),
        .zero_flag   (zero_flag),
        .pc_sel      (pc_sel),
        .flush       (flush)
    );

    // ---------------- ID/EX pipeline register ----------------
    wire [2:0]  ex_alu_ctrl;
    wire        ex_alu_src_sel, ex_wb_sel, ex_reg_we, ex_mem_we;
    wire        ex_zero_we, ex_carry_we;
    wire [1:0]  ex_rd, ex_rs;
    wire [3:0]  ex_pc;
    wire        ex_valid;
    wire [11:0] ex_instr;

    // CALLS: module pipeline_register from file pipeline_register.v
    pipeline_register ID_EX (
        .clk            (clk),
        .rst            (rst),
        .flush          (flush),
        .alu_ctrl_in    (id_alu_ctrl),
        .alu_src_sel_in (id_alu_src_sel),
        .wb_sel_in      (id_wb_sel),
        .reg_we_in      (id_reg_we),
        .mem_we_in      (id_mem_we),
        .zero_we_in     (id_zero_we),
        .carry_we_in    (id_carry_we),
        .is_jmp_in      (id_is_jmp),
        .is_jz_in       (id_is_jz),
        .rd_in          (id_rd),
        .rs_in          (id_rs),
        .imm_in         (id_imm),
        .pc_in          (id_pc),
        .valid_in       (id_valid),
        .instr_in       (id_instr),
        .alu_ctrl       (ex_alu_ctrl),
        .alu_src_sel    (ex_alu_src_sel),
        .wb_sel         (ex_wb_sel),
        .reg_we         (ex_reg_we),
        .mem_we         (ex_mem_we),
        .zero_we        (ex_zero_we),
        .carry_we       (ex_carry_we),
```

```
    .is_jmp         (ex_is_jmp),
    .is_jz          (ex_is_jz),
    .rd             (ex_rd),
    .rs             (ex_rs),
    .imm            (ex_imm),
    .pc             (ex_pc),
    .valid          (ex_valid),
    .instr          (ex_instr)
);

// ================================================================
// STAGE 3: EX - EXECUTE (operand read, ALU, memory, write-back)
// ================================================================
wire [3:0] rd_value;           // qa: value of Rd
wire [3:0] rs_value;           // qb: value of Rs
wire [3:0] alu_b;
wire [3:0] alu_result;
wire       alu_zero, alu_carry;
wire [3:0] mem_data;
wire [3:0] wb_data;

// ---- operand read ----
// CALLS: module register_file from file register_file.v
register_file RF (
    .clk (clk),
    .rst (rst),
    .ra  (ex_rd),              // port A reads Rd
    .rb  (ex_rs),              // port B reads Rs
    .wa  (ex_rd),              // result is written into Rd
    .wd  (wb_data),
    .we  (ex_reg_we),
    .qa  (rd_value),
    .qb  (rs_value)
);

// ---- operand MUX: B = Rs value (0) or immediate (1) ----
// CALLS: module mux2 from file mux.v
mux2 OPERAND_MUX (
    .d0  (rs_value),
    .d1  (ex_imm),
    .sel (ex_alu_src_sel),
    .y   (alu_b)
);

// ---- ALU ----
// CALLS: module alu from file alu.v
alu ALU (
    .a        (rd_value),
    .b        (alu_b),
    .alu_ctrl (ex_alu_ctrl),
    .result   (alu_result),
    .carry    (alu_carry),
    .zero     (alu_zero)
);

// ---- flags ----
// CALLS: module flag_register from file flag_register.v
flag_register FLAGS (
    .clk        (clk),
    .rst        (rst),
    .zero_we    (ex_zero_we),
    .carry_we   (ex_carry_we),
    .zero_in    (alu_zero),
    .carry_in   (alu_carry),
    .zero_flag  (zero_flag),
    .carry_flag (carry_flag)
);
```

```
    // ---- memory access: address = ALU result, data = Rd ----
    // CALLS: module data_memory from file data_memory.v
    data_memory DMEM (
        .clk        (clk),
        .address    (alu_result),
        .write_data (rd_value),
        .mem_we     (ex_mem_we),
        .read_data  (mem_data)
    );

    // ---- write-back MUX: ALU result (0) or memory data (1) ----
    // CALLS: module mux2 from file mux.v
    mux2 WB_MUX (
        .d0  (alu_result),
        .d1  (mem_data),
        .sel (ex_wb_sel),
        .y   (wb_data)
    );

    // ================================================================
    // Debug outputs
    // ================================================================
    assign pc_out      = pc;
    assign instr_out   = ex_instr;
    assign alu_out     = alu_result;
    assign wb_data_out = wb_data;
    assign zero_out    = zero_flag;
    assign carry_out   = carry_flag;

endmodule
```

***Source code: cpu_top.v***

# 6. Testbench and Simulation Results

## 6.1. Testbench Structure

Every testbench is **self-checking** and has the same layout: it prints the **student name and roll number** (`Truong Dong Hung, SE201121`) at the beginning and at the end; **Part 1** shows examples (the demonstration program with a cycle-by-cycle pipeline trace); **Part 2** runs the checks; and the last line is `PASS: all N checks verified, 0 errors` (or `FAIL`).

| Testbench | Unit under test | Checks |
|---|---|---|
| tb_alu | alu: ADD, SUB, AND, OR, XOR, NOT, PASS, carry, zero | 11 |
| tb_register_file | register_file: reset, write, read, two ports | 11 |
| tb_control_unit | control_unit: 16 opcodes and branch unit | 22 |
| tb_cpu | cpu_top: 4 programs, flush, flags, reset | 44 |
| **Total** | | **88** |

*Table 11. Testbenches and number of checks*

## 6.2. Waveforms of the CPU Testbench (F2)

Figures F2a to F2e are Vivado waveform screenshots of `tb_cpu`. Each one shows one group of functions; Table 12 lists the window and the values that must be visible. The setup (signals to add and radix) is given in Appendix A.

| Figure | Function (testbench part) | Window | Values visible |
|---|---|---|---|
| F2a | Pipeline fill, ADD, XOR, JMP flush (TEST 1, program.mem) | 0 to 130 | R1 = 0101 at 45, R2 = 0011 at 55, R1 = 1000 at 65 (ADD), R3 = 1010 at 75, R1 = 0010 at 85 (XOR); flush = 1 at 85 and 115 (JMP 5) |
| F2b | ADD, SUB, carry, zero, STORE, LOAD (TEST 2, prog_arith.mem) | 165 to 265 | R1 = 1000 at 195 (ADD, carry 0); mem_we = 1 at 195 and 225 (STORE); R1 = 0010 and carry_out = 1 at 225 (SUB); R0 = 1000 at 245 (LOAD); R2 = 0000 and zero_out = 1 at 255 |
| F2c | AND, OR, XOR, MOV, NOT, LDI, zero (TEST 3, prog_logic.mem) | 295 to 425 | R3: 1100 (325), 1000 (335, AND), 1100 (345), 1110 (355, OR); R1 = 0110 at 365 (XOR); R2 = 0110 at 375 (MOV); R0 = 1010 at 395 (NOT); R3 = 1010 at 405 (LDI); R3 = 0000 and zero_out = 1 at 415; carry_out stays 0 |
| F2d | JMP flush, JZ taken, JZ not taken (TEST 4, prog_jump.mem) | 455 to 600 | flush = 1 at 465 (JMP 4), 515 (JZ 9 taken), 585 (JMP 13); no flush at 565 to 575 (JZ 14 not taken); R1 = 0000 with zero_out = carry_out = 1 at 515; R3 = 0100 with zero_out = 0 at 565; R2 = 0110 at 585 |
| F2e | Reset and restart (TEST 5) | 595 to 666 | registers and pc_out return to 0 at about 605; the program restarts: R0 = 0001 at 635 |

*Table 12. What each waveform must show (times in ns, from the simulation of tb_cpu)*

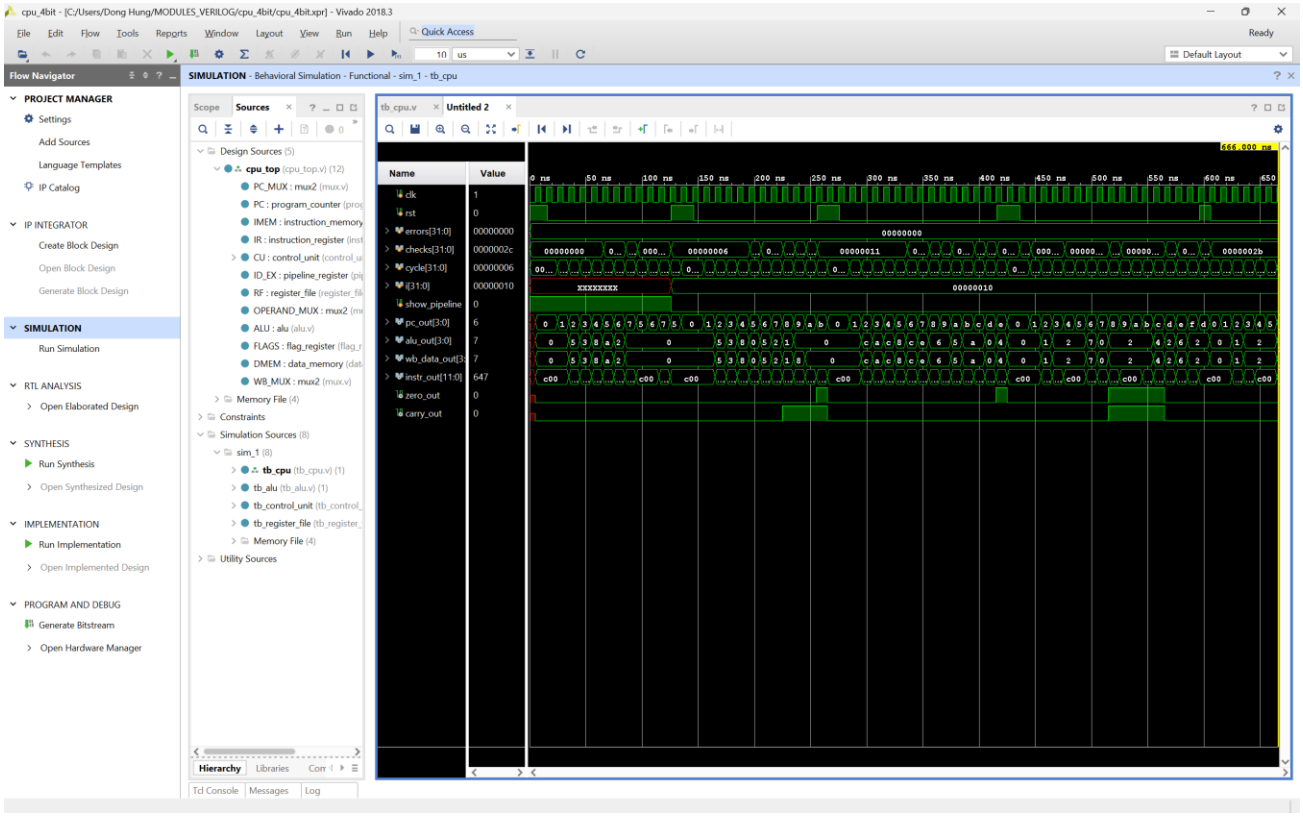



*Figure F2: Waveform.*

# 6. Conclusion

The 4-bit pipelined CPU executes twelve instructions at one instruction per clock cycle, needs no stall for data dependencies, and handles a taken jump with a fixed two-cycle flush. The testbenches print the student name and roll number, run an examples part and a checking part, and end with a PASS line for all 88 checks. The Vivado flow (F1 to F6) shows the sources, the simulation, the elaborated and synthesized schematics and the resource usage. One observation: the data-memory write enable is not gated by reset, which matters only if reset arrives exactly during a STORE; the fix is `.mem_we(ex_mem_we & ~rst)`.